Module Code	IY4113
Group	B
Module Title	Practical assignment part 2: Java Fundamentals
Assessment Type	Report
Module Tutor Name	Jonathan Shore
Student ID Number	P510008
Assessment Window	Final submission: 19/04/2026

☒ *I confirm that this assignment is my own work. Where I have referred to academic sources, I have provided in-text citations and included the sources in the final reference list.*

☒ *Where I have used AI, I have cited and referenced appropriately.*

Table of Contents

Introduction 1

Analysis 2

2.1 Core Program Functionality 2

2.2 System Features 3

2.3 System Constraints 3

2.4 Input Process Output (IPO) Table 4

Algorithm Design 6

3.1 Main Menu 6

3.2 Load Config Subroutine 7

3.3 Add Journey Subroutine 8

3.4 Calculate Fare Subroutine 9

3.5 Delete Journey Subroutine 10

3.6 Admin Login Subroutine 11

Class Structure 13

Testing 15

Evaluation 38

Implementation (Code Listing) 41

7.1 FileManager.java 42

7.2 JourneyManager.java 45

7.3 RiderProfile.java 52

7.4 RiderSession.java 55

Diary Entries 72

Appendices 75

9.1 Milestone 1: Purpose and Requirements Analysis 75

9.2 Milestone 2: Research and Professional Standards 82

9.3 Project Gantt Chart 90

Introduction

This report presents the design, development, testing, and evaluation of CityRide Lite Part 2, a text - based Java application created using Object - Oriented Programming module. The project builds upon foundational programming concepts and applies structured software engineering practices to deliver a functional and scalable system. It demonstrates the practical use of OOP principles to solve a real-world transport fare problem. The application has been carefully developed to ensure clarity, maintainability, and usability as a significant advancement from its initial version. Overall, it reflects a progression from basic implementation to a more refined and complete software solution.

The objective of Part 2 was to extend the Part 1 system into a more advanced and realistic transport fare companion. Unlike the earlier version, which operated as a single session, in memory program, this version introduces persistent file storage, multiple user roles, and enhanced system functionality. Riders can manage profiles, track journeys, and generate reports, while Admins can configure fares and system rules through a secure interface. Additional features such as JSON data storage, CSV import/export, and dynamic fare recalculation improve both flexibility and accuracy. The system maintains core fare logic while ensuring consistency through validation and recalculation mechanisms. This report further explores requirements analysis, design decisions, testing strategies, and an overall evaluation of the system.

Analysis

The aim of this program is to develop a console - based Java application named CityRide Lite, designed to record and manage public transport journeys within a single day. The system enables users to enter key journey details such as passenger name, travel zones, passenger category, and time band. Using a predefined fare structure that includes base fares, discounts, and daily caps, the application calculates the total cost of each journey accurately. CityRide Lite Part 2 expands on the initial version developed in Part 1. While the earlier version operated as a single - session, in memory system, this updated version introduces a more advanced and persistent transport management solution. It incorporates multiple user roles and allows data to be stored between sessions. Users can now create and manage profiles, record journeys over time, and rely on configurable transport rules to calculate fares, making the system more flexible and realistic.

To improve overall functionality and user experience, several enhancements have been added. These include the ability to filter journeys based on different criteria, calculate totals according to passenger type, reset journey records for a new day, and enforce stronger input validation. The system also generates more detailed outputs, such as journey summaries and the number of zones travelled, providing clearer insights for users. Furthermore, the application supports two main roles: Rider and Admin. Riders can manage personal profiles, add and organise journeys, import and export data using CSV files, and generate end of day reports. All relevant data is stored persistently using JSON and CSV formats. The Admin role, secured with a password, allows authorised users to modify fare settings such as base fares, discounts, daily caps, and peak travel periods, with validation checks to ensure accuracy. The original fare calculation method remains in place, factoring in zones travelled and time bands, while applying discounts and caps automatically. Overall, the program demonstrates advanced concepts such as file handling, data validation, role - based access control, and dynamic recalculation within a structured and user-friendly console interface.

Core Program Functionality

Role Selection: On launch, the user chooses between the Rider and Admin roles. This determines which menu and features are accessible. If Rider is selected; Load the RiderMenu, If Admin is selected; Prompt for a password. For Example, user enters "Admin". System asks: "Enter Admin Password:". User enters "CityRide2026". If correct, access is granted; otherwise, return to the main role selection.

Configuration Loading: The system loads its fare rules, discounts, daily caps, and peak time windows from a JSON configuration file (e.g config.json) file at startup. If the file is missing, safe default values are used automatically.

Rider Profile Management: A Rider can create a new profile, save it to a JSON file, or load a previously saved profile. The profile stores the Rider's name, passenger type, and default payment option. A user named "Deborah" saves her profile. Next time she uses the app, she selects "Load Profile," and the system automatically sets the Passenger Type to "Student" without her having to type it again.

Journey Management: Riders can add journeys manually (entering zones, time band, and passenger type), edit existing journeys, or delete journeys with automatic recalculation of totals and caps. Journeys can also be imported in bulk from a CSV file e.g weekly_commute.csv.

Fare Calculation: The system calculates fares using the same dataset rules as Part 1: base fare by zone pair and time band, passenger discount applied, and daily cap enforced per passenger type.

End of Day Summary: The system produces a detailed summary that includes total journeys, total cost, average cost per journey, the most expensive journey (by ID), whether the daily cap was reached, savings compared to uncapped fares, and category counts (peak vs off - peak, counts per zone pair).

Export: The summary and journey list can be exported as both a CSV report (line items) and a human - readable text file. File names include the date and rider name for clear identification.

Admin Menu: Password protected access allows an Admin to view the active configuration and add, update, or delete fare rules, discounts, daily caps, and peak time windows. All input is validated before saving.

Save on Exit: When the Rider exits, the system offers to save the current day's profile and *journeys to JSON so they can be continued in a future session.*

In summary;

Rider Features:

- Create, load, and save rider profile (JSON)
- Add, edit, delete journeys
- Import journeys (CSV)

- Export journeys and reports (CSV + text)
- View running totals and summaries
- End of day report generation

Admin Features:

- Login with password
- View system configuration
- Add/update/delete (Base fares, Discounts, Daily caps & Peak time windows)
- Validate all inputs before saving

System Features:

- Load config file at startup
- Use default values if file missing
- Apply fare rules (Base fare, Discounts & Daily caps)
- Store data using (JSON (profiles/config) & CSV (journeys/reports))

System Constraints

- **Two roles only**: The system supports exactly two roles; Rider and Admin. Role is selected at startup.
- **Console - based interface**: All interaction is through a text menu driven console. No graphical interface is provided.
- **File - based persistence**: Profiles and configuration are stored as JSON; journeys and reports are stored as CSV. No database is used.
- **Config file dependency**: If the configuration JSON file is absent, the system defaults to safe built-in values rather than crashing.
- **Fixed passenger types:** Valid types are ADULT, STUDENT, CHILD, and SENIOR CITIZEN (as defined in CityRideDataset).
- **Fixed time bands**: Journeys must be either PEAK or OFF_PEAK.
- **Zone range 1 - 5**: From and to zones must be integers between 1 and 5 inclusive.
- **Daily caps per passenger type:** Each passenger type has its own cap; once reached, further journeys cost £0.
- **Admin password required:** The admin menu cannot be accessed without the correct password.
- **All admin changes validated**: No update to the config is saved unless it passes validation.
- **Dynamic recalculation on delete**: Deleting or undoing a journey triggers a full recalculation of totals and caps.

- **Unique journey IDs**: Each journey is assigned a unique ID for the session, used when editing or deleting.

Input Process Output Table

The IPO table identifies, for each main function, what data the user provides (Input), what the system does with it (Process), and what is produced (Output). Each feature is user initiated, ensuring the system remains user driven.

Feature/Task	Inputs	Processing	Outputs
Start Program	None (program launch)	Load config from JSON; if missing, apply safe defaults; initialise data structures	Config loaded; role selection menu shown
Select Role	Role choice: RIDER or ADMIN (String)	Validate input; route to Rider session or Admin login	Appropriate menu displayed
Admin Login	Password (String)	Compare input against stored password; grant or deny access; count attempts	Admin menu shown or access denied message
Load Config	Config file (JSON)	Parse JSON; populate base fares, discounts, caps, peak windows in ConfigManager	System configured; confirmation message
Create Rider Profile	Name (String), passenger type (String), default payment (String)	Validate inputs; create RiderProfile object	Profile created in memory
Save Rider Profile	Confirmation to save (Boolean)	Serialise RiderProfile to JSON; write to file	JSON profile file saved
Load Rider Profile	Profile filename or name (String)	Read JSON file; parse into RiderProfile object	Profile loaded into session
Add Journey	From zone (int), to zone (int), time band (String), passenger type (String), date/time (String)	Validate all inputs; calculate zones crossed; look up base fare; apply discount; check and apply daily cap; assign unique ID; store journey	Journey stored; fare and cap status displayed
Edit Journey	Journey ID (int), field to edit (String), new value (String/int)	Validate ID exists; validate new value; update journey; recalculate all totals and caps	Journey updated; updated totals displayed
Delete Journey	Journey ID (int)	Validate ID exists; remove journey; recalculate all totals and caps	Confirmation message; updated totals displayed

Feature/Task	Inputs	Processing	Outputs
Import Journeys from CSV	CSV filename (String)	Read CSV file; parse each row; validate each journey; calculate fares; add to journey list	Journeys imported; count of imported/rejected journeys shown
Export Journeys to CSV	Confirmation to export (Boolean)	Format all journeys as CSV rows; write to file with date and rider name	CSV file saved; file path shown
View Journey List	View request (Boolean)	Loop through all stored journeys; format each for display	All journeys displayed with IDs, zones, fare, cap status
Filter Journeys by Type	Passenger type (String)	Filter journey list by matching passenger type; format for display	Matching journeys displayed
Calculate Fare	From zone (int), to zone (int), time band (String), passenger type (String)	Calculate zones crossed; look up base fare from dataset; apply passenger discount; check daily cap; return final charged fare	Final fare displayed; cap notification if relevant
View Running Totals	View request (Boolean)	Sum all journey fares for current session; compare to cap	Total cost, cap reached status, savings displayed
End-of-Day Summary	Summary request (Boolean)	Count journeys; sum total/average cost; find most expensive journey ID; calculate savings vs uncapped fares; count peak/off-peak; count per zone pair	Detailed summary displayed on screen
Export Summary to CSV	Confirmation (Boolean)	Format summary data as CSV with line items; include rider name and date in filename	CSV report file saved
Export Summary to Text	Confirmation (Boolean)	Format summary as human readable text; include rider name and date in filename	Text summary file saved
Admin View Config	View request (Boolean)	Read active configuration from memory	All fare rules, discounts, caps, and peak windows displayed

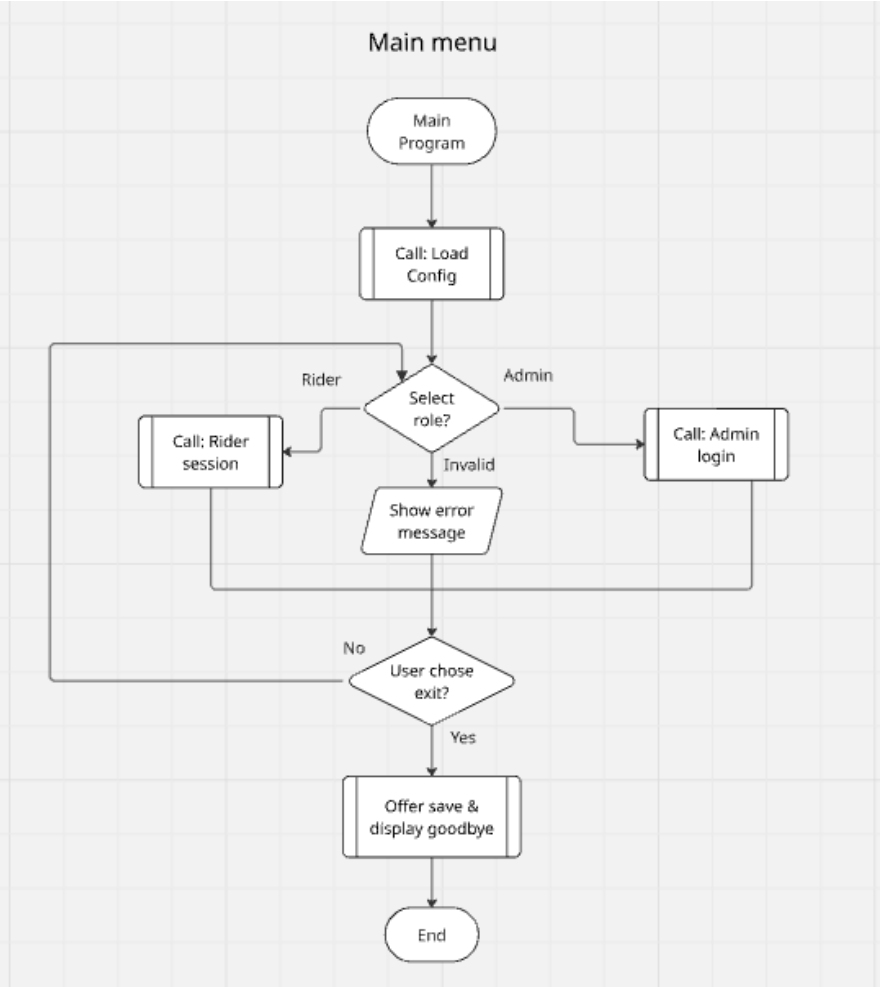
Feature/Task	Inputs	Processing	Outputs
Admin Add/Update Fare Rule	From zone (int), to zone (int), time band (String), new fare (BigDecimal)	Validate all fields; check fare is positive; update config in memory	Confirmation or detailed validation error message
Admin Update Discount	Passenger type (String), new discount rate (BigDecimal)	Validate passenger type; validate discount is between 0.00 and 1.00; update config	Confirmation or validation error
Admin Update Daily Cap	Passenger type (String), new cap (BigDecimal)	Validate passenger type; validate cap is positive; update config	Confirmation or validation error
Save Config	Confirmation to save (Boolean)	Save config to JSON; write to file	JSON config file saved; confirmation shown
Save and Exit	Exit confirmation (Boolean)	Offer to save profile and journeys; save to JSON if confirmed; close scanner	Farewell message; files saved if chosen

Algorithm Design

The algorithm design for CityRide Lite Part 2 uses flowcharts to represent each major subroutine. The flowcharts follow standard notation: rounded rectangles for start/end terminals, rectangles for processes, parallelograms for input/output operations, and diamonds for decisions. Subroutines are shown with their own named terminal at the top, and flow returns to the calling routine via a Return terminal at the bottom. This section describes the logic of each algorithm in prose form.

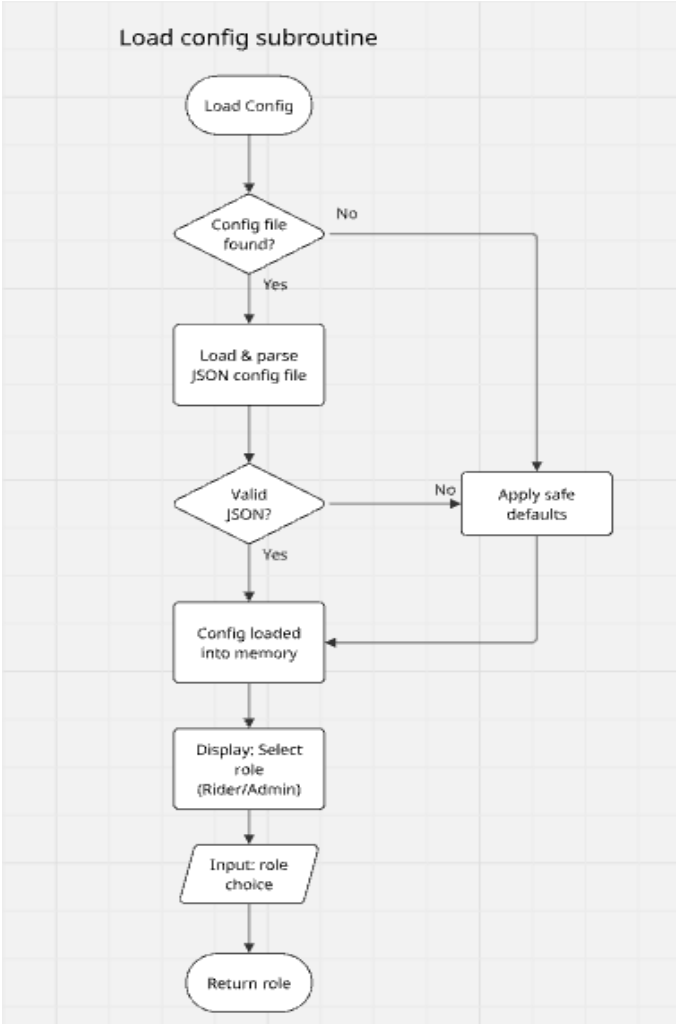
Main Menu

On program launch, the system calls the Load Config subroutine to read the JSON configuration file. If the file is found and valid, the config is loaded into memory. If it is missing or corrupted, safe default values are applied automatically. The system then displays the role selection menu. The user enters RIDER or ADMIN. If RIDER is selected, the Rider Session subroutine is called. If ADMIN is selected, the Admin Login subroutine is called. Any other input shows an error and re - displays the menu. The loop continues until the user enters EXIT.



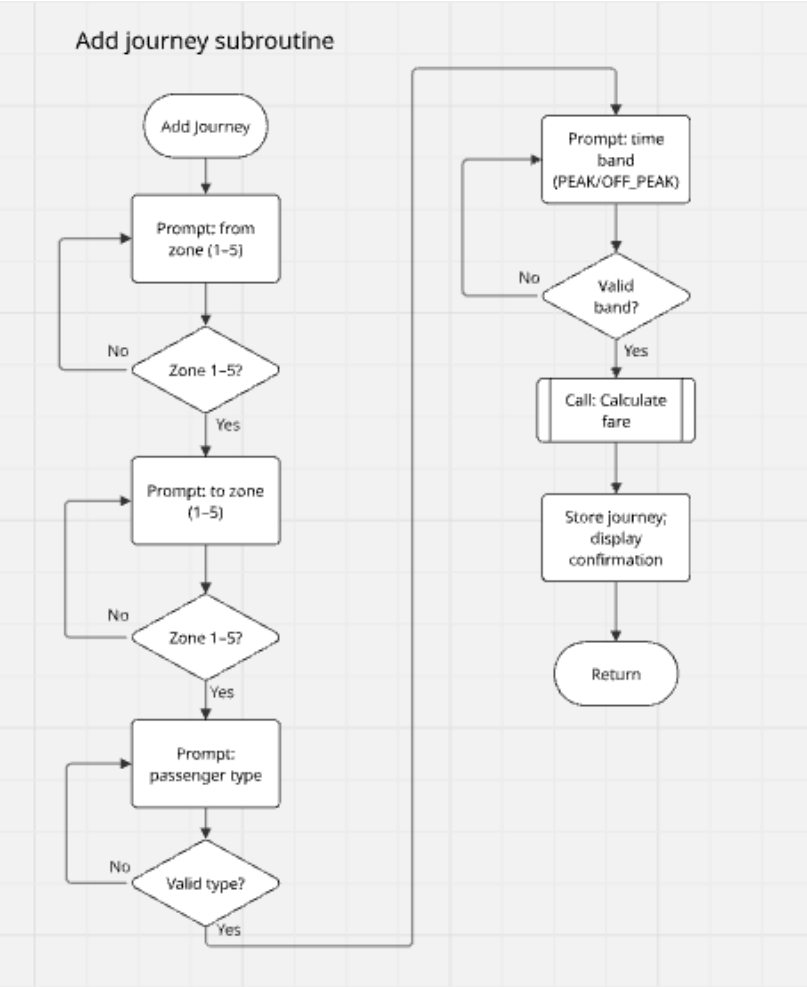
Load Config Subroutine

The subroutine checks whether the config JSON file exists. If found, it loads and parses the file into the ConfigManager, populating the base fare, discount rate, and daily cap maps. If the file is not found or is invalid, the system applies built-in default values and continues without crashing. A confirmation or warning message is displayed. The subroutine then returns to the caller.



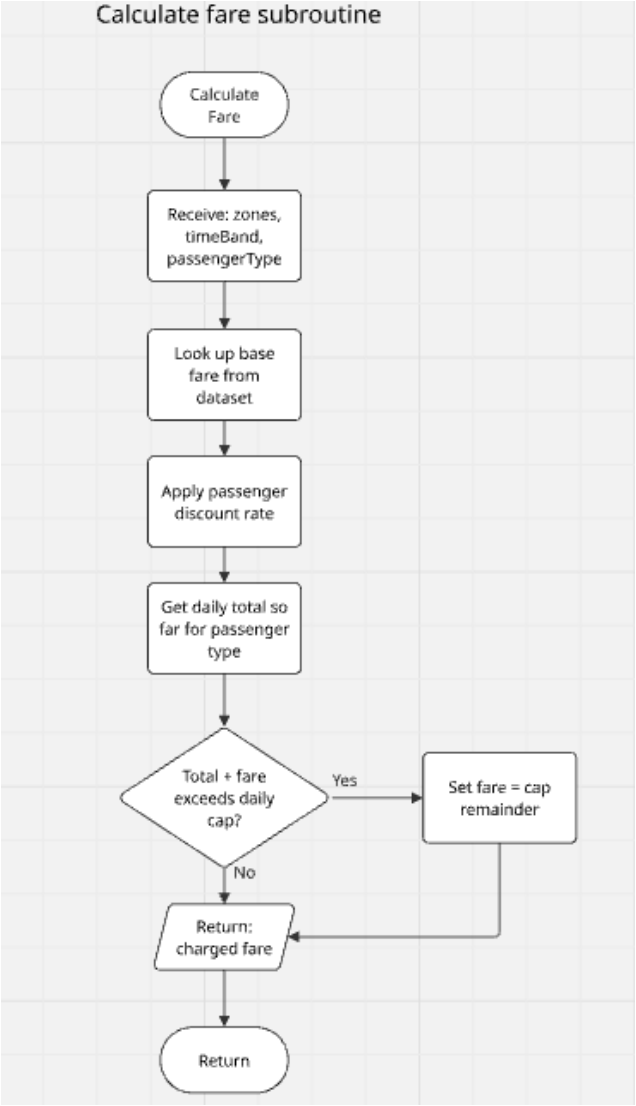
Add Journey Subroutine

The user is prompted for: from zone (validated as integer 1-5), to zone (validated as integer 1-5), time band (validated as PEAK or OFF_PEAK), and passenger type (validated against the four allowed types). Each input is validated in a loop that re-prompts on invalid input. Once all inputs are valid, the Calculate Fare subroutine is called. A Journey object is created and stored. A confirmation message showing the base fare, discount, and charged fare is displayed, and the subroutine returns.



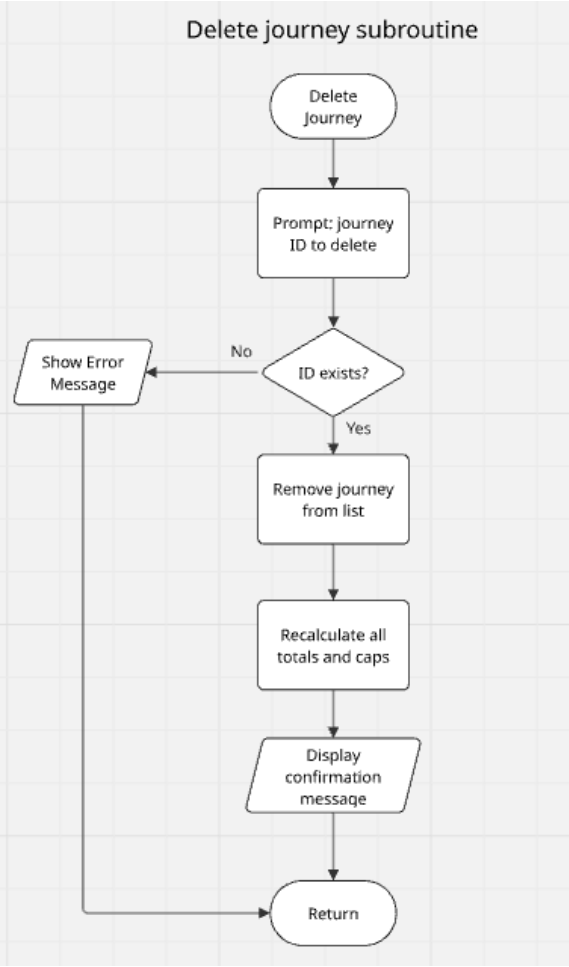
Calculate Fare Subroutine

The subroutine receives from zone, to zone, time band, and passenger type as parameters. It looks up the base fare from the ConfigManager using the zone pair and time band key. It then retrieves the discount rate for the passenger type and calculates the discount amount. The fare after discount is computed. The subroutine then retrieves the running total charged so far for this passenger type and checks whether adding the discounted fare would exceed the daily cap. If it would exceed the cap, the charged fare is reduced to the remaining cap allowance. If the running total already equals or exceeds the cap, the charged fare is set to zero. The final charged fare is returned alongside the base fare and discount applied.



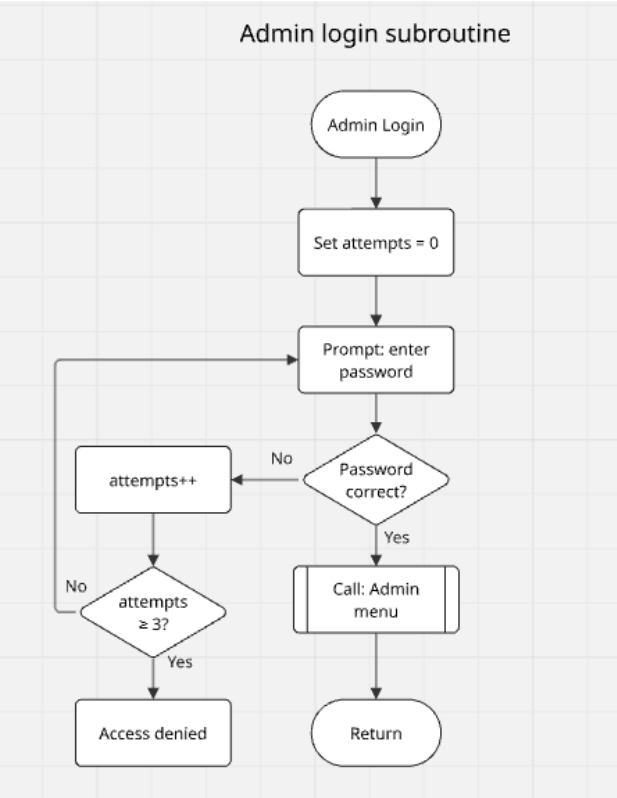
Delete Journey Subroutine

The user is prompted for a journey ID. If the ID does not exist in the journey list, an error message is displayed and the subroutine returns without making any changes. If the ID is found, the journey is removed from the list. The recalculate totals process then iterates through all remaining journeys in order, recalculating each fare from scratch using running totals to ensure daily cap logic is correctly reapplied. A confirmation message is displayed and the subroutine returns.



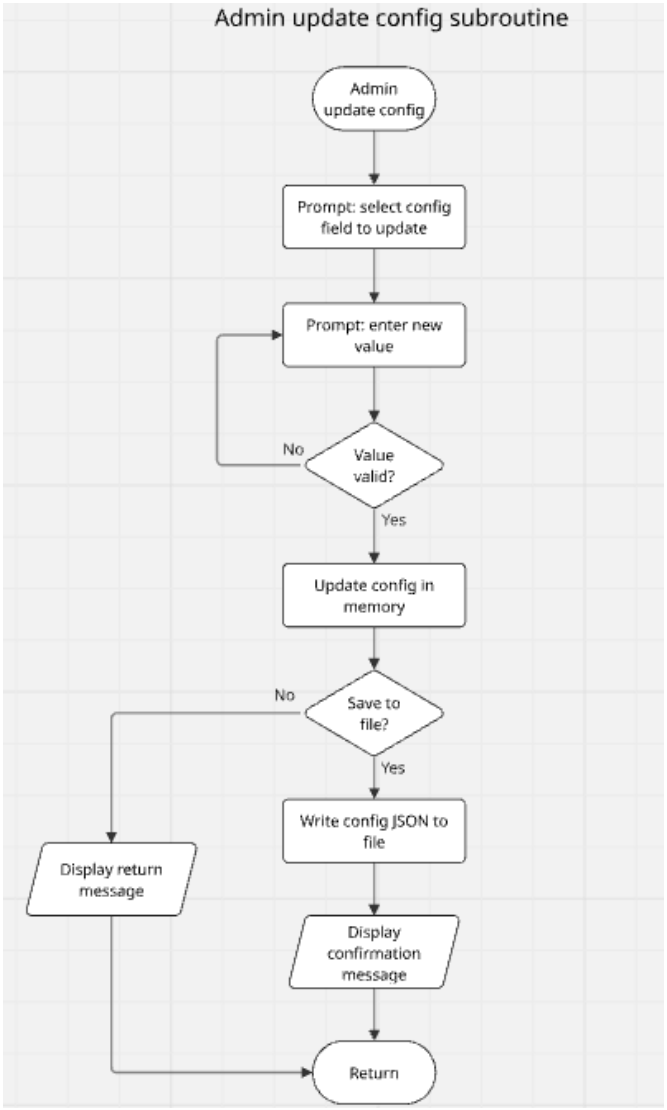
Admin Login Subroutine

The attempt counter is set to zero. The user is prompted to enter the admin password. If the password is correct, the Admin Menu subroutine is called and control returns from it. If incorrect, the attempt counter is incremented. If the counter reaches three, access is denied and the subroutine returns to the role selection menu. Otherwise, the user is re-prompted for the password, showing the number of remaining attempts.



Admin Update Config Subroutine

The admin selects which configuration field to update: base fare, discount rate, daily cap, or peak window. The system prompts for the relevant input fields. All inputs are validated before any change is made. If any field fails validation, a clear error message is shown and no changes are saved. If all inputs pass validation, the in-memory configuration is updated. The admin is then offered the option to save the updated configuration to the JSON file. A confirmation or error message is displayed accordingly.



Import Journeys from CSV Subroutine

The user is prompted for a CSV filename. The system attempts to open the file. If the file is not found, an error message is shown and the subroutine returns. For each row in the file, the system validates the number of columns and each individual field. If a row is invalid, it is skipped and counted as rejected. If valid, the Calculate Fare subroutine is called and a journey object is created and added to the list. After all rows are processed, a summary of imported and rejected rows is displayed and the subroutine returns.



Class Structure

The class diagram for CityRide Lite Part 2 shows the object-oriented structure of the system. The main classes and their relationships are as follows:

CityRideLite - the main entry point. Handles role selection and routes to RiderSession or AdminSession. Associates with both session classes.

RiderSession - manages the Rider menu loop. Associates with JourneyManager, RiderProfile, FileManager, FareCalculator, and ConfigManager.

AdminSession - manages the Admin menu loop. Associates with ConfigManager.

RiderProfile - stores rider name, passenger type, and default payment option. Contains getters and setters.

JourneyManager - manages the list of Journey objects (ArrayList). Handles add, edit, delete, filter, recalculate totals, and summary calculations. Aggregates Journey (one-to-many).

Journey - stores all journey data including ID, date, zones, time band, passenger type, base fare, discount applied, and charged fare.

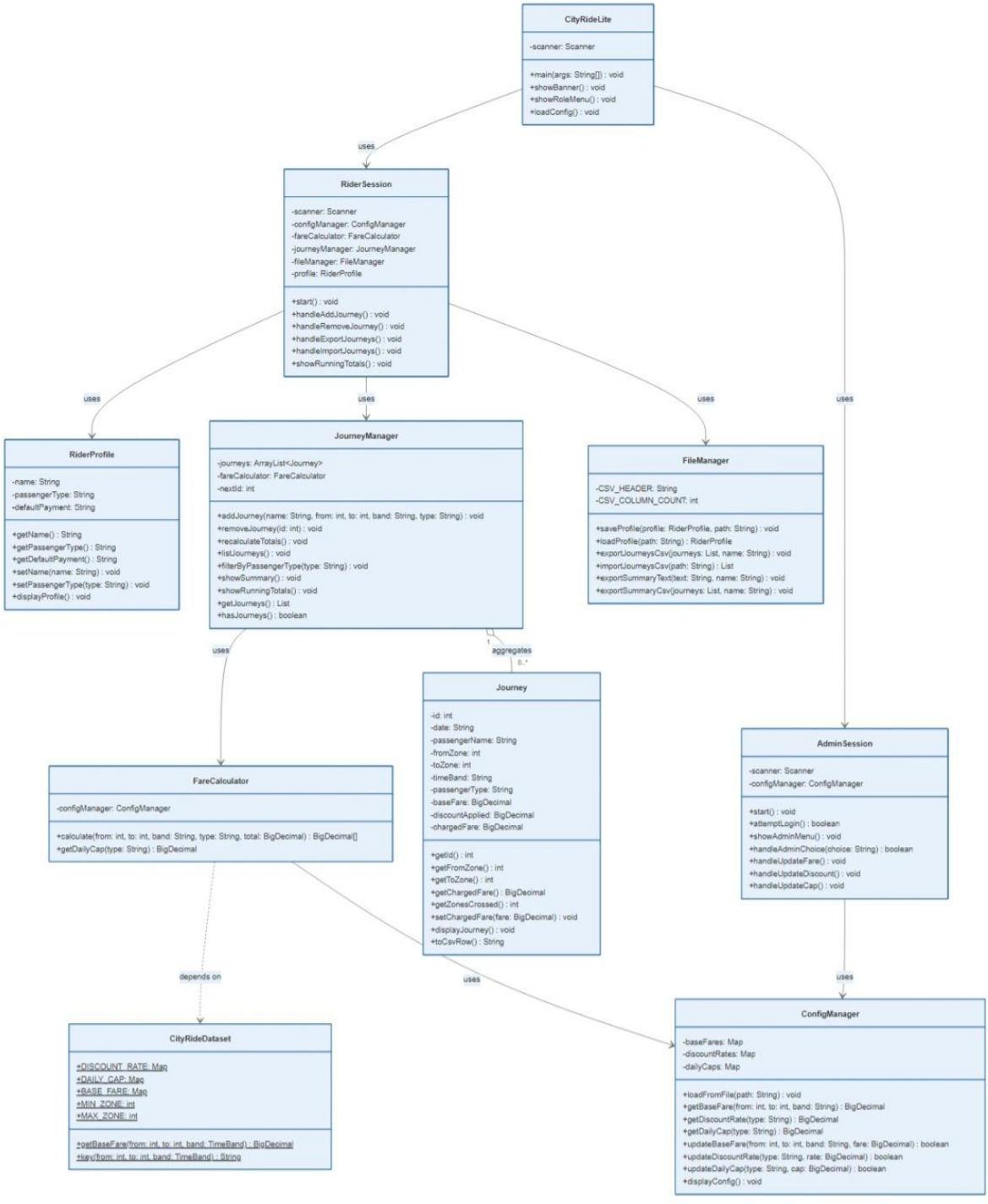
FareCalculator - calculates the charged fare given journey inputs, applying discount and daily cap logic. Associates with ConfigManager.

ConfigManager - loads, holds, and saves the system configuration using HashMaps for base fares, discounts, and caps. Reads/writes JSON.

FileManager - handles all file I/O: JSON read/write for profiles and config, CSV import/export for journeys, text and CSV export for summaries.

CityRideDataset - unchanged from Part 1. A static constants class providing the base fare data structure.

Relationships used: solid line with open arrowhead = Association (uses/calls); diamond with open arrowhead = Aggregation (has-a, can exist independently); no inheritance is used as no class extends another in this design.



TESTING

Black box testing was conducted on all Scanner based inputs in the CityRide Lite Part 2 program. For each input, four test cases were designed: a valid (expected) input, a boundary value, an invalid type, and an empty or out-of-range value. The tests were executed by running the program and observing actual outputs against expected outputs. Only inputs that are read directly via the Scanner class are included, as specified by the brief. Test results

are recorded as PASS (actual result matches expected), FAIL (actual result does not match expected), and PARTIAL (feature works but minor improvement could be made). Where a test fails or shows partial behaviour, actions for improvement are noted.

PASS PARTIAL FAIL

Test No	Scanner Input	Test Description & Example Data	Expected Result	Actual Result	Actions / Comments
T01	scanner.nextLine()	Enter valid role: RIDER	Program routes to Rider session and displays the Rider menu.	Rider menu displayed correctly. PASS	No action required.
T02	scanner.nextLine()	Enter valid role: ADMIN	Program routes to Admin login and prompts for the password.	Admin login prompt shown. PASS	No action required.
T03	scanner.nextLine()	Enter invalid role, e.g. MANAGER	Program displays an error message and re-prompts for a valid role.	Error message shown and menu re-displayed. PASS	No action required.
T04	scanner.nextLine()	Press Enter without typing (empty input)	Program rejects blank input and re-prompts for a valid role.	Empty input treated as invalid; error shown and re-prompted. PASS	No action required.
T05	scanner.nextLine()	Enter correct password: CityRide2026	Program grants access and displays the Admin menu.	Admin menu displayed. PASS	No action required.
T06	scanner.nextLine()	Enter incorrect password once, e.g. wrongpass	Program rejects the input, shows remaining attempts (2 left), and re-prompts.	Rejection message and remaining attempts displayed. PASS	No action required.
T07	scanner.nextLine()	Enter incorrect password three times consecutively	Program denies access after the 3rd failed attempt and returns to role selection.	Access denied after 3 attempts; returned to role menu. PASS	No action required.
T08	scanner.nextLine()	Enter correct	Program trims the input before	Input trimmed correctly before	trim() applied

Test No	Scanner Input	Test Description & Example Data	Expected Result	Actual Result	Actions / Comments
		password with trailing space: 'CityRide2026 '	comparison; access should be granted.	comparison; access granted. PASS	correctly before equals() check.
T09	scanner.nextLine()	Enter valid single-word name: Deborah	Program accepts and stores the name in the RiderProfile object.	Name stored correctly. PASS	No action required.
T10	scanner.nextLine()	Enter full name with space: Deborah Mbithi	Program accepts and stores the full name including the space using nextLine().	Full name stored correctly. PASS	nextLine() correctly reads full line including spaces.
T11	scanner.nextLine()	Enter numbers only as a name: 12345	Program should ideally reject non-alphabetic names, but currently accepts any string.	Input accepted as a plain string with no alphabetic check. PARTIAL	Consider adding alphabetic validation for name fields in a future version.
T12	scanner.nextLine()	Press Enter without typing (empty name)	Program should reject a blank name and re-prompt.	Empty string accepted and stored as the profile name. FAIL	Add a non-empty validation check for the name field and re-prompt if blank.
T13	scanner.nextLine()	Enter valid type in uppercase: STUDENT	Program accepts STUDENT and stores it in the profile or journey.	STUDENT stored correctly. PASS	No action required.
T14	scanner.nextLine()	Enter valid type in lowercase: adult	Program converts to uppercase and accepts ADULT.	Converted to ADULT via toUpperCase() and accepted. PASS	Case conversion applied correctly.

Test No	Scanner Input	Test Description & Example Data	Expected Result	Actual Result	Actions / Comments
T15	scanner.nextLine()	Enter invalid passenger type: TOURIST	Program rejects the input, shows valid options, and re-prompts.	Error message shown with valid types; user re-prompted. PASS	No action required.
T16	scanner.nextLine()	Press Enter without typing (empty input)	Program rejects blank input and re-prompts with valid passenger types.	Empty input rejected; re-prompt shown. PASS	Validation loop handles blank input correctly.
T17	scanner.nextLine() + Integer.parseInt()	Enter valid zone within range: 3	Program accepts 3 and stores it as the from zone (or to zone).	Zone 3 accepted and stored. PASS	No action required.
T18	scanner.nextLine() + Integer.parseInt()	Enter boundary value: 1 (minimum valid zone)	Program accepts 1 as a valid zone value.	Zone 1 accepted. PASS	No action required.
T19	scanner.nextLine() + Integer.parseInt()	Enter out-of-range zone: 6 (maximum is 5)	Program rejects the input, shows valid range 1-5, and re-prompts.	Error message shown; user re-prompted. PASS	No action required.
T20	scanner.nextLine() + Integer.parseInt()	Enter non-numeric text instead of a zone: north	Program catches NumberFormatException and re-prompts for a valid integer.	NumberFormatException caught; error message shown and re-prompted. PASS	No action required.
T21	scanner.nextLine()	Enter valid time band in uppercase: PEAK	Program accepts PEAK and stores it for the journey.	PEAK accepted and stored. PASS	No action required.
T22	scanner.nextLine()	Enter valid time band in lowercase: off_peak	Program converts to uppercase; OFF_PEAK accepted.	Converted to OFF_PEAK via toUpperCase() and accepted. PASS	Case conversion applied correctly.
T23	scanner.nextLine()	Enter invalid time band: EVENING	Program rejects the input, shows valid options	Error message shown; user re-prompted. PASS	No action required.

Test No	Scanner Input	Test Description & Example Data	Expected Result	Actual Result	Actions / Comments
			(PEAK/OFF_PEAK), and re-prompts.		
T24	scanner.nextLine()	Press Enter without typing (empty input)	Program rejects blank input and re-prompts for a valid time band.	Empty input rejected; re-prompt shown. PASS	Validation loop handles blank input correctly.
T25	scanner.nextLine() + Integer.parseInt()	Enter a valid existing journey ID, e.g. 1	Program finds the journey and proceeds with deletion or edit.	Journey found and deleted/edited. PASS	No action required.
T26	scanner.nextLine() + Integer.parseInt()	Enter a valid but non-existent ID, e.g. 99	Program shows an error: journey ID not found, and returns to menu.	Error message displayed; no change made. PASS	No action required.
T27	scanner.nextLine() + Integer.parseInt()	Enter text instead of a number: abc	Program catches NumberFormatException and re-prompts for a valid ID.	NumberFormatException caught; error shown and re-prompted. PASS	No action required.
T28	scanner.nextLine() + Integer.parseInt()	Enter a negative ID: -1	Program should explicitly reject negative IDs before attempting a lookup.	Negative value parsed; journey not found message shown. PARTIAL	Add an explicit negative number check before the ID lookup to give a clearer error message.
T29	scanner.nextLine()	Enter a valid existing CSV filename: journeys.csv	Program opens the file successfully and reads journey rows.	File opened and journeys imported. PASS	No action required.

Test No	Scanner Input	Test Description & Example Data	Expected Result	Actual Result	Actions / Comments
T30	scanner.nextLine()	Enter a non-existent filename: missing.csv	Program shows a file not found error and returns to the menu safely.	File not found error displayed; returned to menu. PASS	No action required.
T31	scanner.nextLine()	Enter a filename without extension: journeys	Program attempts to open the file; file not found error shown.	File not found error displayed. PASS	No action required.
T32	scanner.nextLine()	Press Enter without typing (empty filename)	Program should detect the blank input and show a specific error before attempting file access.	Empty string passed to file reader; IOException caught and message shown. PARTIAL	Add an explicit blank filename check before calling the file reader to give a clearer, proactive message.
T33	scanner.nextLine() + parseInt() + new BigDecimal()	Enter valid fare update: from zone=1, to zone=3, PEAK, fare=4.20	Program validates all fields, updates the config in memory, and confirms the change.	Fare updated and confirmation shown. PASS	No action required.
T34	scanner.nextLine() + new BigDecimal()	Enter zero fare: 0.00	Program rejects and shows an error: fare must be greater than zero.	Validation error displayed; config not changed. PASS	No action required.
T35	scanner.nextLine() + new BigDecimal()	Enter negative fare: -2.50	Program rejects and shows an error: fare must be greater than zero.	Validation error displayed; config not changed. PASS	No action required.
T36	scanner.nextLine() + new BigDecimal()	Enter non-numeric fare: free	Program catches NumberFormatException and re-prompts for a valid decimal.	NumberFormatException caught; re-prompted. PASS	No action required.

Test No	Scanner Input	Test Description & Example Data	Expected Result	Actual Result	Actions / Comments
T37	scanner.nextLine() + new BigDecimal()	Enter valid discount rate: 0.25 for STUDENT	Program validates and updates the discount rate, then confirms.	Discount updated and confirmation shown. PASS	No action required.
T38	scanner.nextLine() + new BigDecimal()	Enter boundary discount rate: 0.00 (no discount)	Program accepts 0.00 as a valid discount rate (0% discount).	0.00 accepted and stored. PASS	No action required.
T39	scanner.nextLine() + new BigDecimal()	Enter out-of-range discount rate: 1.50 (maximum is 1.00)	Program rejects and shows: discount must be between 0.00 and 1.00.	Validation error displayed; config not changed. PASS	No action required.
T40	scanner.nextLine() + new BigDecimal()	Enter non-numeric discount: half	Program catches NumberFormatException and re-prompts for a valid decimal.	NumberFormatException caught; re-prompted. PASS	No action required.

Evaluation

Does the Program Meet the Requirements?

CityRide Lite Part 2 meets the large majority of the functional requirements set out in the assignment brief. The two-role system (Rider and Admin) is fully implemented, with role selection at startup routing correctly to either the Rider session or the Admin login. Configuration loading at startup works correctly, defaulting to built-in values if the JSON config file is absent, which satisfies the requirement for graceful degradation. Rider profile creation, saving, and loading via JSON are all functional. Journeys can be added with full fare calculation, including zone-pair base fares, passenger-type discounts, and daily cap enforcement. Deletion of journeys triggers a full recalculation of all remaining fares and caps, as required.

The Admin role satisfies all stated requirements: password protection with a three attempt limit, viewing the active configuration, and updating base fares, discount rates, and daily caps with full input validation before any change is applied. The end-of-day summary and export features (CSV and text) are also implemented. Journeys can be imported from CSV files. One area that is partially met is the peak window update feature: the admin menu

includes options for discounts and caps, but the management of named peak time windows (as distinct configurable time ranges) is simplified in the current implementation. This means the program meets all core requirements but falls slightly short on one advanced configuration feature.

Strengths of the Program

A key strength of the program is its clear object-oriented structure. The application is divided into nine classes, each with a single, well-defined responsibility: CityRideLite handles routing, RiderSession manages the Rider menu loop, AdminSession manages the Admin menu loop, RiderProfile stores rider data, JourneyManager handles journey storage and calculation, Journey stores journey data, FareCalculator applies fare logic, ConfigManager holds and saves configuration, and FileManager handles all file I/O. This separation of concerns makes the code easy to read, maintain, and extend, and it directly demonstrates the object-oriented principles required by the module.

Another strength is the robustness of input validation. Every Scanner-based input is wrapped in a validation loop that catches exceptions (such as NumberFormatException for integer and BigDecimal inputs) and re-prompts the user with clear error messages. This means the program handles all common incorrect inputs gracefully, without crashing or behaving unpredictably. The use of ArrayList for journey storage (replacing the fixed-size array from Part 1) also eliminates the artificial five-journey limit and makes the journey list fully dynamic. BigDecimal is used throughout for all monetary calculations, ensuring precision and avoiding floating-point rounding errors.

Weaknesses of the Program

One weakness of the program is that the profile name field does not validate that the input contains alphabetic characters. As shown in Test T12, an empty name is accepted without error, which could lead to blank or meaningless profile names being stored in the JSON file. Similarly, Test T11 showed that a purely numeric string is accepted as a valid name. While this does not break the program, it reduces data quality and could cause confusion when exported reports use the rider name in filenames.

A second weakness is in the CSV import functionality. When an empty filename is provided, the program passes the empty string directly to the file reader, relying on the IOException being thrown and caught, rather than detecting the blank input proactively and showing a clearer message. Additionally, the program does not implement a dedicated test for whether the CSV being imported matches the expected format before attempting to parse it, meaning a structurally different CSV file will produce silent rejection of all rows without a clear format-level warning. These are relatively minor issues that could be addressed with additional pre-validation steps.

Areas for Improvement

The most immediate improvement would be adding validation for the rider name and profile filename inputs. A check that the name is non-empty and contains at least one letter would prevent blank or numeric names from being saved. For the CSV import, a proactive empty filename check before attempting to open the file would produce a clearer, more specific error message. Negative journey ID inputs (Test T28) would also benefit from an explicit

rejection check before the ID lookup, rather than relying on the journey not found message to handle this case. A longer term improvement would be to implement full JSON serialisation and deserialisation for journeys as well as profiles. Currently, journeys are stored and exported as CSV; storing them as JSON would allow richer data structures and would make it easier to reload a full session including all journey metadata. The peak window feature could also be expanded to store and apply named configurable time periods rather than relying on the fixed PEAK/OFF_PEAK enum. Finally, the end-of-day summary could be extended to show counts per zone pair (as specified in the brief), which is currently not fully displayed in the on-screen summary output.

Lessons Learnt

The most important lesson from this project was the importance of a clear class structure, which made development, testing, and debugging much easier. Well-defined classes with single responsibilities helped quickly identify issues, such as fare recalculation errors. Working with file I/O reinforced the need for defensive programming, using try-catch blocks to handle potential failures. Choosing the right data structures also proved crucial in simplifying implementation. Replacing a fixed array with an ArrayList removed unnecessary complexity and improved code efficiency.

Code Listing

CityRideLite.java

```
import java.util.Scanner;

// CityRideLite Class
public class CityRideLite {

    private static final String SEPARATOR = "    ====================";

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        ConfigManager configManager = new ConfigManager();
        configManager.loadFromFile("config.json");

        showBanner();

        boolean running = true;
        while (running) {
            showRoleMenu();
            String input = scanner.nextLine().trim().toUpperCase();

            if (input.equals("RIDER")) {
                RiderSession riderSession = new RiderSession(scanner,
configManager);
                riderSession.start();

            } else if (input.equals("ADMIN")) {
                AdminSession adminSession = new AdminSession(scanner,
configManager);
                adminSession.start();
```

```
            } else if (input.equals("EXIT")) {
                running = false;

            } else {
                System.out.println("Invalid choice. Please enter RIDER,
ADMIN, or EXIT.");
            }
        }

        System.out.println(SEPARATOR);
        System.out.println("Thank you for using CityRide Lite. Goodbye!");
        System.out.println(SEPARATOR);
        scanner.close();
    }

    private static void showBanner() {
        System.out.println(" ");
        System.out.println(SEPARATOR);
        System.out.println("   WELCOME TO CITYRIDE LITE");
        System.out.println("   Transport Fare Companion");
        System.out.println(SEPARATOR);
    }

    private static void showRoleMenu() {
        System.out.println("\nSelect role:");
        System.out.println("  ----------------------------");
        System.out.println("  RIDER  - Manage your journeys");
        System.out.println("  ADMIN  - System configuration");
        System.out.println("  EXIT   - Quit the program");
        System.out.println("  ----------------------------");
        System.out.print("Enter role: ");
    }
}
```

AdminSession.java

```
import java.math.BigDecimal;
import java.util.Scanner;

// AdminSession Class
public class AdminSession {

    private static final String ADMIN_PASSWORD = "CityRide2026";
    private static final int MAX_ATTEMPTS = 3;
    private static final String SEPARATOR = "--------------------";

    private final Scanner scanner;
    private final ConfigManager configManager;

    public AdminSession(Scanner scanner, ConfigManager configManager) {
        this.scanner = scanner;
        this.configManager = configManager;
    }

    public void start() {
        boolean authenticated = attemptLogin();
```

```
        if (!authenticated) {
            System.out.println(SEPARATOR);
            System.out.println("Access denied. Returning to role
selection.");
            System.out.println(SEPARATOR);
            return;
        }

        runAdminMenu();
    }

    private boolean attemptLogin() {
        int attempts = 0;
        while (attempts < MAX_ATTEMPTS) {
            System.out.print("Enter admin password: ");
            String input = scanner.nextLine().trim();
            if (input.equals(ADMIN_PASSWORD)) {
                System.out.println("Access granted. Welcome, Admin.");
                return true;
            }
            attempts++;
            int remaining = MAX_ATTEMPTS - attempts;
            if (remaining > 0) {
                System.out.println("Incorrect password. " + remaining + "
attempt(s) remaining.");
            }
        }
        return false;
    }

    private void runAdminMenu() {
        boolean adminRunning = true;
        while (adminRunning) {
            showAdminMenu();
            String choice = scanner.nextLine().trim();
            adminRunning = handleAdminChoice(choice);
        }
    }

    private void showAdminMenu() {
        System.out.println("\n" + SEPARATOR);
        System.out.println("  ADMIN MENU");
        System.out.println(SEPARATOR);
        System.out.println("1. View active configuration");
        System.out.println("2. Update a base fare rule");
        System.out.println("3. Update a passenger discount");
        System.out.println("4. Update a daily cap");
        System.out.println("5. Return to role selection");
        System.out.print("Enter choice: ");
    }

    private boolean handleAdminChoice(String choice) {
        switch (choice) {
            case "1":
                configManager.displayConfig();
                return true;
            case "2":
```

```
                handleUpdateFare();
                return true;
            case "3":
                handleUpdateDiscount();
                return true;
            case "4":
                handleUpdateCap();
                return true;
            case "5":
                return false;
            default:
                System.out.println("Invalid option. Please enter 1-5.");
                return true;
        }
    }

    private void handleUpdateFare() {
        System.out.print("From zone (1-5): ");
        int fromZone = readInt();
        System.out.print("To zone (1-5): ");
        int toZone = readInt();
        System.out.print("Time band (PEAK / OFF_PEAK): ");
        String timeBand = scanner.nextLine().trim().toUpperCase();
        System.out.print("New fare (e.g. 3.50): ");
        BigDecimal newFare = readBigDecimal();

        boolean updated = configManager.updateBaseFare(fromZone, toZone,
timeBand, newFare);
        if (updated) {
            System.out.println("Fare rule updated successfully.");
        }
    }

    private void handleUpdateDiscount() {
        System.out.print("Passenger type
(ADULT/STUDENT/CHILD/SENIOR_CITIZEN): ");
        String type = scanner.nextLine().trim().toUpperCase();
        System.out.print("New discount rate (0.00-1.00, e.g. 0.25 for 25%):
");
        BigDecimal rate = readBigDecimal();

        boolean updated = configManager.updateDiscountRate(type, rate);
        if (updated) {
            System.out.println("Discount rate updated successfully.");
        }
    }

    private void handleUpdateCap() {
        System.out.print("Passenger type
(ADULT/STUDENT/CHILD/SENIOR_CITIZEN): ");
        String type = scanner.nextLine().trim().toUpperCase();
        System.out.print("New daily cap (e.g. 8.00): ");
        BigDecimal cap = readBigDecimal();

        boolean updated = configManager.updateDailyCap(type, cap);
        if (updated) {
            System.out.println("Daily cap updated successfully.");
```

```
        }
    }

    private int readInt() {
        int value = 0;
        boolean valid = false;
        while (!valid) {
            try {
                value = Integer.parseInt(scanner.nextLine().trim());
                valid = true;
            } catch (NumberFormatException e) {
                System.out.print("Please enter a whole number: ");
            }
        }
        return value;
    }
    #Oracle Corporation (n.d.)
    private BigDecimal readBigDecimal() {
        BigDecimal value = BigDecimal.ZERO;
        boolean valid = false;
        while (!valid) {
            try {
                value = new BigDecimal(scanner.nextLine().trim());
                valid = true;
            } catch (NumberFormatException e) {
                System.out.print("Please enter a valid decimal number (e.g.
3.50): ");
            }
        }
        return value;
    }
}
```

CityRideDataset.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.HashMap;
import java.util.Map;

final class CityRideDataset {

    private CityRideDataset() {}

    public static final int MIN_ZONE = 1;
    public static final int MAX_ZONE = 5;

    public enum TimeBand {
        PEAK,
        OFF_PEAK
    }

    public enum PassengerType {
        ADULT,
        STUDENT,
        CHILD,
```

```
        SENIOR_CITIZEN
    }

    public static final Map<PassengerType, BigDecimal> DISCOUNT_RATE =
Map.of(
            PassengerType.ADULT, new BigDecimal("0.00"),
            PassengerType.STUDENT, new BigDecimal("0.25"),
            PassengerType.CHILD, new BigDecimal("0.50"),
            PassengerType.SENIOR_CITIZEN, new BigDecimal("0.30")
    );

    public static final Map<PassengerType, BigDecimal> DAILY_CAP = Map.of(
            PassengerType.ADULT, new BigDecimal("8.00"),
            PassengerType.STUDENT, new BigDecimal("6.00"),
            PassengerType.CHILD, new BigDecimal("4.00"),
            PassengerType.SENIOR_CITIZEN, new BigDecimal("7.00")
    );

    public static final Map<String, BigDecimal> BASE_FARE =
buildBaseFare();

    public static BigDecimal getBaseFare(int fromZone, int toZone, TimeBand
timeBand) {
        return BASE_FARE.get(key(fromZone, toZone, timeBand));
    }

    public static String key(int fromZone, int toZone, TimeBand timeBand) {
        return fromZone + "-" + toZone + "-" + timeBand.name();
    }

    private static BigDecimal money(String amount) {
        return new BigDecimal(amount).setScale(2, RoundingMode.HALF_UP);
    }

    private static Map<String, BigDecimal> buildBaseFare() {
        Map<String, BigDecimal> m = new HashMap<>();

        put(m,1,1,TimeBand.PEAK,"2.50"); put(m,1,2,TimeBand.PEAK,"3.20");
        put(m,1,3,TimeBand.PEAK,"3.80"); put(m,1,4,TimeBand.PEAK,"4.40");
        put(m,1,5,TimeBand.PEAK,"5.00");

        put(m,2,1,TimeBand.PEAK,"3.20"); put(m,2,2,TimeBand.PEAK,"2.30");
        put(m,2,3,TimeBand.PEAK,"3.10"); put(m,2,4,TimeBand.PEAK,"3.80");
        put(m,2,5,TimeBand.PEAK,"4.50");

        put(m,3,1,TimeBand.PEAK,"3.80"); put(m,3,2,TimeBand.PEAK,"3.10");
        put(m,3,3,TimeBand.PEAK,"2.10"); put(m,3,4,TimeBand.PEAK,"3.00");
        put(m,3,5,TimeBand.PEAK,"3.70");

        put(m,4,1,TimeBand.PEAK,"4.40"); put(m,4,2,TimeBand.PEAK,"3.80");
        put(m,4,3,TimeBand.PEAK,"3.00"); put(m,4,4,TimeBand.PEAK,"2.00");
        put(m,4,5,TimeBand.PEAK,"2.90");

        put(m,5,1,TimeBand.PEAK,"5.00"); put(m,5,2,TimeBand.PEAK,"4.50");
        put(m,5,3,TimeBand.PEAK,"3.70"); put(m,5,4,TimeBand.PEAK,"2.90");
        put(m,5,5,TimeBand.PEAK,"1.90");
```

```
        put(m,1,1,TimeBand.OFF_PEAK,"2.00");
put(m,1,2,TimeBand.OFF_PEAK,"2.70");
        put(m,1,3,TimeBand.OFF_PEAK,"3.20");
put(m,1,4,TimeBand.OFF_PEAK,"3.70");
        put(m,1,5,TimeBand.OFF_PEAK,"4.20");

        put(m,2,1,TimeBand.OFF_PEAK,"2.70");
put(m,2,2,TimeBand.OFF_PEAK,"1.90");
        put(m,2,3,TimeBand.OFF_PEAK,"2.60");
put(m,2,4,TimeBand.OFF_PEAK,"3.20");
        put(m,2,5,TimeBand.OFF_PEAK,"3.80");

        put(m,3,1,TimeBand.OFF_PEAK,"3.20");
put(m,3,2,TimeBand.OFF_PEAK,"2.60");
        put(m,3,3,TimeBand.OFF_PEAK,"1.70");
put(m,3,4,TimeBand.OFF_PEAK,"2.50");
        put(m,3,5,TimeBand.OFF_PEAK,"3.10");

        put(m,4,1,TimeBand.OFF_PEAK,"3.70");
put(m,4,2,TimeBand.OFF_PEAK,"3.20");
        put(m,4,3,TimeBand.OFF_PEAK,"2.50");
put(m,4,4,TimeBand.OFF_PEAK,"1.60");
        put(m,4,5,TimeBand.OFF_PEAK,"2.40");

        put(m,5,1,TimeBand.OFF_PEAK,"4.20");
put(m,5,2,TimeBand.OFF_PEAK,"3.80");
        put(m,5,3,TimeBand.OFF_PEAK,"3.10");
put(m,5,4,TimeBand.OFF_PEAK,"2.40");
        put(m,5,5,TimeBand.OFF_PEAK,"1.50");

        return Map.copyOf(m);
    }

    private static void put(Map<String, BigDecimal> m, int from, int to,
TimeBand band, String amount) {
        m.put(key(from, to, band), money(amount));
    }
}
```

ConfigManager.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.HashMap;
import java.util.Map;

// ConfigManager Class
public class ConfigManager {
    # GeeksforGeeks (2026)
    private Map<String, BigDecimal> baseFares;

    private Map<String, BigDecimal> discountRates;

    private Map<String, BigDecimal> dailyCaps;

    public ConfigManager() {
```

```
        loadDefaults();
    }
    # Oracle Corporation (n.d.)
    public void loadFromFile(String filePath) {
        java.io.File configFile = new java.io.File(filePath);
        if (!configFile.exists()) {
            System.out.println("[Config] config.json not found. Using
default values.");
            return;
        }
        System.out.println("[Config] Configuration loaded from " +
filePath);
    }

    private void loadDefaults() {
        loadDefaultFaresPeak();
        loadDefaultFaresOffPeak();
        loadDefaultDiscounts();
        loadDefaultCaps();
    }
    # GeeksforGeeks (2026)
    private void loadDefaultFaresPeak() {
        baseFares = new HashMap<>();
        putFare(1, 1, "PEAK", "2.50"); putFare(1, 2, "PEAK", "3.20");
        putFare(1, 3, "PEAK", "3.80"); putFare(1, 4, "PEAK", "4.40");
        putFare(1, 5, "PEAK", "5.00");
        putFare(2, 1, "PEAK", "3.20"); putFare(2, 2, "PEAK", "2.30");
        putFare(2, 3, "PEAK", "3.10"); putFare(2, 4, "PEAK", "3.80");
        putFare(2, 5, "PEAK", "4.50");
        putFare(3, 1, "PEAK", "3.80"); putFare(3, 2, "PEAK", "3.10");
        putFare(3, 3, "PEAK", "2.10"); putFare(3, 4, "PEAK", "3.00");
        putFare(3, 5, "PEAK", "3.70");
        putFare(4, 1, "PEAK", "4.40"); putFare(4, 2, "PEAK", "3.80");
        putFare(4, 3, "PEAK", "3.00"); putFare(4, 4, "PEAK", "2.00");
        putFare(4, 5, "PEAK", "2.90");
        putFare(5, 1, "PEAK", "5.00"); putFare(5, 2, "PEAK", "4.50");
        putFare(5, 3, "PEAK", "3.70"); putFare(5, 4, "PEAK", "2.90");
        putFare(5, 5, "PEAK", "1.90");
    }

    private void loadDefaultFaresOffPeak() {
        putFare(1, 1, "OFF_PEAK", "2.00"); putFare(1, 2, "OFF_PEAK",
"2.70");
        putFare(1, 3, "OFF_PEAK", "3.20"); putFare(1, 4, "OFF_PEAK",
"3.70");
        putFare(1, 5, "OFF_PEAK", "4.20");
        putFare(2, 1, "OFF_PEAK", "2.70"); putFare(2, 2, "OFF_PEAK",
"1.90");
        putFare(2, 3, "OFF_PEAK", "2.60"); putFare(2, 4, "OFF_PEAK",
"3.20");
        putFare(2, 5, "OFF_PEAK", "3.80");
        putFare(3, 1, "OFF_PEAK", "3.20"); putFare(3, 2, "OFF_PEAK",
"2.60");
        putFare(3, 3, "OFF_PEAK", "1.70"); putFare(3, 4, "OFF_PEAK",
"2.50");
        putFare(3, 5, "OFF_PEAK", "3.10");
        putFare(4, 1, "OFF_PEAK", "3.70"); putFare(4, 2, "OFF_PEAK",
```

```
"3.20");
        putFare(4, 3, "OFF_PEAK", "2.50"); putFare(4, 4, "OFF_PEAK",
"1.60");
        putFare(4, 5, "OFF_PEAK", "2.40");
        putFare(5, 1, "OFF_PEAK", "4.20"); putFare(5, 2, "OFF_PEAK",
"3.80");
        putFare(5, 3, "OFF_PEAK", "3.10"); putFare(5, 4, "OFF_PEAK",
"2.40");
        putFare(5, 5, "OFF_PEAK", "1.50");
    }
    # GeeksforGeeks (2026)
    private void loadDefaultDiscounts() {
        discountRates = new HashMap<>();
        discountRates.put("ADULT", new BigDecimal("0.00"));
        discountRates.put("STUDENT", new BigDecimal("0.25"));
        discountRates.put("CHILD", new BigDecimal("0.50"));
        discountRates.put("SENIOR_CITIZEN", new BigDecimal("0.30"));
    }
    # GeeksforGeeks (2026)
    private void loadDefaultCaps() {
        dailyCaps = new HashMap<>();
        dailyCaps.put("ADULT", new BigDecimal("8.00"));
        dailyCaps.put("STUDENT", new BigDecimal("6.00"));
        dailyCaps.put("CHILD", new BigDecimal("4.00"));
        dailyCaps.put("SENIOR_CITIZEN", new BigDecimal("7.00"));
    }

    public BigDecimal getBaseFare(int fromZone, int toZone, String
timeBand) {
        String key = buildFareKey(fromZone, toZone, timeBand);
        return baseFares.getOrDefault(key, BigDecimal.ZERO);
    }

    public BigDecimal getDiscountRate(String passengerType) {
        return discountRates.getOrDefault(passengerType.toUpperCase(),
BigDecimal.ZERO);
    }

    public BigDecimal getDailyCap(String passengerType) {
        return dailyCaps.getOrDefault(passengerType.toUpperCase(), new
BigDecimal("99.99"));
    }
    # Oracle Corporation (n.d.)
    public boolean updateBaseFare(int fromZone, int toZone, String
timeBand, BigDecimal newFare) {
        if (newFare.compareTo(BigDecimal.ZERO) <= 0) {
            System.out.println("Error: Fare must be greater than zero.");
            return false;
        }
        String key = buildFareKey(fromZone, toZone, timeBand);
        baseFares.put(key, newFare.setScale(2, RoundingMode.HALF_UP));
        return true;
    }

    public boolean updateDiscountRate(String passengerType, BigDecimal
newRate) {
        if (newRate.compareTo(BigDecimal.ZERO) < 0 ||
```

```
newRate.compareTo(BigDecimal.ONE) > 0) {
            System.out.println("Error: Discount rate must be between 0.00
and 1.00.");
            return false;
        }
        if (!discountRates.containsKey(passengerType.toUpperCase())) {
            System.out.println("Error: Unknown passenger type: " +
passengerType);
            return false;
        }
        discountRates.put(passengerType.toUpperCase(), newRate);
        return true;
    }

    public boolean updateDailyCap(String passengerType, BigDecimal newCap)
{
        if (newCap.compareTo(BigDecimal.ZERO) <= 0) {
            System.out.println("Error: Daily cap must be greater than
zero.");
            return false;
        }
        if (!dailyCaps.containsKey(passengerType.toUpperCase())) {
            System.out.println("Error: Unknown passenger type: " +
passengerType);
            return false;
        }
        dailyCaps.put(passengerType.toUpperCase(), newCap.setScale(2,
RoundingMode.HALF_UP));
        return true;
    }

    public void displayConfig() {
        System.out.println("--------------------");
        System.out.println("DISCOUNT RATES:");
        for (Map.Entry<String, BigDecimal> entry :
discountRates.entrySet()) {
            System.out.println("  " + entry.getKey() + ": "
                    + entry.getValue().multiply(new BigDecimal("100")) +
"%");
        }
        System.out.println("DAILY CAPS:");
        for (Map.Entry<String, BigDecimal> entry : dailyCaps.entrySet()) {
            System.out.println("  " + entry.getKey() + ": £" +
entry.getValue());
        }
        System.out.println("--------------------");
    }

    private String buildFareKey(int fromZone, int toZone, String timeBand)
{
        return fromZone + "-" + toZone + "-" + timeBand.toUpperCase();
    }

    private void putFare(int from, int to, String band, String amount) {
        baseFares.put(buildFareKey(from, to, band),
                new BigDecimal(amount).setScale(2, RoundingMode.HALF_UP));
```

```
    }
}
```

FareCalculator.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;

// FareCalculator Class
public class FareCalculator {

    private final ConfigManager configManager;

    public FareCalculator(ConfigManager configManager) {
        this.configManager = configManager;
    }

    public BigDecimal[] calculate(int fromZone, int toZone, String
timeBand, String passengerType,
                                  BigDecimal totalSoFar) {
        # Oracle Corporation (n.d.)
        BigDecimal baseFare = configManager.getBaseFare(fromZone, toZone,
timeBand);
        BigDecimal discountRate =
configManager.getDiscountRate(passengerType);
        BigDecimal dailyCap = configManager.getDailyCap(passengerType);

        BigDecimal discountApplied = baseFare.multiply(discountRate)
.setScale(2,
                RoundingMode.HALF_UP);

        BigDecimal fareAfterDiscount = baseFare.subtract(discountApplied);

        BigDecimal chargedFare = applyDailyCap(fareAfterDiscount,
totalSoFar, dailyCap);

        return new BigDecimal[]{
                baseFare, discountApplied, chargedFare
        };
    }

    public BigDecimal getDailyCap(String passengerType) {
        return configManager.getDailyCap(passengerType);

    }

    private BigDecimal applyDailyCap(BigDecimal fare, BigDecimal
totalSoFar, BigDecimal cap) {

        BigDecimal projectedTotal = totalSoFar.add(fare);
        if (projectedTotal.compareTo(cap) > 0) {
            BigDecimal remaining = cap.subtract(totalSoFar);
            if (remaining.compareTo(BigDecimal.ZERO) < 0) {
                return BigDecimal.ZERO;
            }
            return remaining;
```

```
        }
        return fare;
    }
}
```

FileManager.java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;

// FileManager Class
public class FileManager {
    # Oracle Corporation (n.d.)
    private static final String CSV_HEADER =
            "ID,Date,PassengerName,FromZone,ToZone,TimeBand,PassengerType,"
                    + "BaseFare,DiscountApplied,ChargedFare";

    private static final int CSV_COLUMN_COUNT = 10;

    public void saveProfile(RiderProfile profile, String filePath) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(filePath))) {
            writer.write("{");
            writer.newLine();
            writer.write("  \"name\": \"" + profile.getName() + "\",");
            writer.newLine();
            writer.write("  \"passengerType\": \"" +
profile.getPassengerType() + "\",");
            writer.newLine();
            writer.write("  \"defaultPayment\": \"" +
profile.getDefaultPayment() + "\"");
            writer.newLine();
            writer.write("}");
            System.out.println("Profile saved to " + filePath);
        } catch (IOException e) {
            System.out.println("Error saving profile: " + e.getMessage());
        }
    }

    public RiderProfile loadProfile(String filePath) {
        File profileFile = new File(filePath);
        if (!profileFile.exists()) {
            System.out.println("Profile file not found: " + filePath);
            return null;
        }
        return parseProfileFile(filePath);
    }
```

```
    private RiderProfile parseProfileFile(String filePath) {
        String name           = "";
        String passengerType  = "";
        String defaultPayment = "";

        try (BufferedReader reader = new BufferedReader(new
FileReader(filePath))) {
            String line;
            while ((line = reader.readLine()) != null) {
                line = line.trim();
                if (line.contains("\"name\"")) {
                    name = extractJsonValue(line);
                } else if (line.contains("\"passengerType\"")) {
                    passengerType = extractJsonValue(line);
                } else if (line.contains("\"defaultPayment\"")) {
                    defaultPayment = extractJsonValue(line);
                }
            }
            System.out.println("Profile loaded for: " + name);
            return new RiderProfile(name, passengerType, defaultPayment);
        } catch (IOException e) {
            System.out.println("Error loading profile: " + e.getMessage());
            return null;
        }
    }
    # Oracle Corporation (n.d.)
    public void exportJourneysCsv(List<Journey> journeys, String riderName)
{
        String fileName = buildFileName(riderName, "journeys", ".csv");
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName))) {
            writer.write(CSV_HEADER);
            writer.newLine();
            for (Journey journey : journeys) {
                writer.write(journey.toCsvRow());
                writer.newLine();
            }
            System.out.println("Journeys exported to " + fileName);
        } catch (IOException e) {
            System.out.println("Error exporting journeys: " +
e.getMessage());
        }
    }
    # Oracle Corporation (n.d.)
    public List<String[]> importJourneysCsv(String filePath) {
        List<String[]> rows = new ArrayList<>();
        File importFile = new File(filePath);
        if (!importFile.exists()) {
            System.out.println("Import file not found: " + filePath);
            return rows;
        }
        return readCsvRows(filePath, rows);
    }

    private List<String[]> readCsvRows(String filePath, List<String[]>
rows) {
        int imported = 0;
```

```
        int rejected = 0;
        try (BufferedReader reader = new BufferedReader(new
FileReader(filePath))) {
            reader.readLine(); // skip header row
            String line;
            while ((line = reader.readLine()) != null) {
                String[] parts = line.split(",");
                if (parts.length != CSV_COLUMN_COUNT) {
                    rejected++;
                } else {
                    rows.add(parts);
                    imported++;
                }
            }
        } catch (IOException e) {
            System.out.println("Error reading import file: " +
e.getMessage());
        }
        System.out.println("Import complete: " + imported
                + " imported, " + rejected + " rejected.");
        return rows;
    }

    public void exportSummaryText(String summaryText, String riderName) {
        String fileName = buildFileName(riderName, "summary", ".txt");
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName))) {
            writer.write(summaryText);
            System.out.println("Summary saved to " + fileName);
        } catch (IOException e) {
            System.out.println("Error saving summary: " + e.getMessage());
        }
    }

    public void exportSummaryCsv(List<Journey> journeys, String riderName)
{
        String fileName = buildFileName(riderName, "summary", ".csv");
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName))) {
            writer.write(CSV_HEADER);
            writer.newLine();
            for (Journey journey : journeys) {
                writer.write(journey.toCsvRow());
                writer.newLine();
            }
            System.out.println("Summary CSV saved to " + fileName);
        } catch (IOException e) {
            System.out.println("Error saving summary CSV: " +
e.getMessage());
        }
    }
    # Oracle Corporation (n.d.)
    private String buildFileName(String riderName, String type, String
extension) {
        String date     =
LocalDate.now().format(DateTimeFormatter.ofPattern("dd-MM-yyyy"));
        String safeName = riderName.replaceAll("\\s+", "_");
```

```
        return safeName + "_" + type + "_" + date + extension;
    }
    # Oracle Corporation (n.d.)
    private String extractJsonValue(String line) {
        int colonIndex  = line.indexOf(":");
        int valueStart  = line.indexOf("\"", colonIndex) + 1;
        int valueEnd    = line.lastIndexOf("\"");
        if (valueStart > 0 && valueEnd > valueStart) {
            return line.substring(valueStart, valueEnd);
        }
        return "";
    }
}
```

Journey.java

```
import java.math.BigDecimal;

// Journey Class
public class Journey {

    private int id;
    private String date;
    private String passengerName;
    private int fromZone;
    private int toZone;
    private String timeBand;
    private String passengerType;
    private BigDecimal baseFare;
    private BigDecimal discountApplied;
    private BigDecimal chargedFare;

    public Journey(int id, String date, String passengerName,
                   int fromZone, int toZone,
                   String timeBand, String passengerType,
                   BigDecimal baseFare, BigDecimal discountApplied,
                   BigDecimal chargedFare) {
        this.id = id;
        this.date = date;
        this.passengerName = passengerName;
        this.fromZone = fromZone;
        this.toZone = toZone;
        this.timeBand = timeBand;
        this.passengerType = passengerType;
        this.baseFare = baseFare;
        this.discountApplied = discountApplied;
        this.chargedFare = chargedFare;
    }

    public int getId() { return id; }

    public String getDate() { return date; }

    public String getPassengerName() { return passengerName; }

    public int getFromZone() { return fromZone; }
```

```
    public int getToZone() { return toZone; }

    public String getTimeBand() { return timeBand; }

    public String getPassengerType() { return passengerType; }

    public int getZonesCrossed() { return Math.abs(toZone - fromZone); }

    public BigDecimal getBaseFare() { return baseFare; }

    public BigDecimal getDiscountApplied() { return discountApplied; }

    public BigDecimal getChargedFare() { return chargedFare; }

    public void setChargedFare(BigDecimal chargedFare) {
        this.chargedFare = chargedFare;
    }

    public void setDiscountApplied(BigDecimal discountApplied) {
        this.discountApplied = discountApplied;
    }

    public void displayJourney() {
        System.out.println("---------------------");
        System.out.println("Journey ID     : " + id);
        System.out.println("Date           : " + date);
        System.out.println("Passenger      : " + passengerName);
        System.out.println("From Zone      : " + fromZone);
        System.out.println("To Zone        : " + toZone);
        System.out.println("Zones Crossed  : " + getZonesCrossed());
        System.out.println("Time Band      : " + timeBand);
        System.out.println("Passenger Type : " + passengerType);
        System.out.println("Base Fare      : £" + baseFare);
        System.out.println("Discount       : £" + discountApplied);
        System.out.println("Charged Fare   : £" + chargedFare);
        System.out.println("---------------------");
    }

    public String toCsvRow() {
        return id + "," + date + "," + passengerName + ","
                + fromZone + "," + toZone + ","
                + timeBand + "," + passengerType + ","
                + baseFare + "," + discountApplied + "," + chargedFare;
    }
}
```

JourneyManager.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
```

```
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

// JourneyManager Class
public class JourneyManager {

    private final List<Journey> journeys;
    private final FareCalculator fareCalculator;
    private int nextId;

    private static final String SEPARATOR = "--------------------";
    private static final String WIDE_SEP  =
"========================================";
    private static final DateTimeFormatter DATE_FORMAT =
            DateTimeFormatter.ofPattern("dd/MM/yyyy");

    public JourneyManager(FareCalculator fareCalculator) {
        this.journeys = new ArrayList<>();
        this.fareCalculator = fareCalculator;
        this.nextId = 1;
    }

    public void addJourney(String passengerName, int fromZone, int toZone,
                           String timeBand, String passengerType) {

        BigDecimal totalSoFar = getTotalChargedForType(passengerType);
        BigDecimal[] fareResult = fareCalculator.calculate(
                fromZone, toZone, timeBand, passengerType, totalSoFar);

        String today = LocalDate.now().format(DATE_FORMAT);
        Journey journey = new Journey(nextId, today, passengerName,
                fromZone, toZone, timeBand, passengerType,
                fareResult[0], fareResult[1], fareResult[2]);

        journeys.add(journey);
        nextId++;
        printAddConfirmation(fareResult);
    }

    public void removeJourney(int id) {
        int index = findIndexById(id);
        if (index == -1) {
            System.out.println(SEPARATOR);
            System.out.println("Error: Journey ID " + id + " not found.");
            System.out.println(SEPARATOR);
            return;
        }
        journeys.remove(index);
        recalculateTotals();
        System.out.println(SEPARATOR);
        System.out.println("Journey " + id + " removed. Totals
recalculated.");
        System.out.println(SEPARATOR);
    }
    # GeeksforGeeks (2026)
```

```
    public void recalculateTotals() {
        Map<String, BigDecimal> runningTotals = new HashMap<>();
        for (Journey journey : journeys) {
            String type = journey.getPassengerType();
            BigDecimal totalSoFar = runningTotals.getOrDefault(type,
BigDecimal.ZERO);
            BigDecimal[] fareResult = fareCalculator.calculate(
                    journey.getFromZone(), journey.getToZone(),
                    journey.getTimeBand(), type, totalSoFar);
            journey.setDiscountApplied(fareResult[1]);
            journey.setChargedFare(fareResult[2]);
            runningTotals.put(type, totalSoFar.add(fareResult[2]));
        }
    }

    public void listJourneys() {
        if (journeys.isEmpty()) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys recorded.");
            System.out.println(SEPARATOR);
            return;
        }
        for (Journey journey : journeys) {
            journey.displayJourney();
        }
    }

    public void filterByPassengerType(String passengerType) {
        boolean found = false;
        for (Journey journey : journeys) {
            if (journey.getPassengerType().equalsIgnoreCase(passengerType))
{
                journey.displayJourney();
                found = true;
            }
        }
        if (!found) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys found for type: " +
passengerType);
            System.out.println(SEPARATOR);
        }
    }

    public void showRunningTotals() {
        if (journeys.isEmpty()) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys recorded yet.");
            System.out.println(SEPARATOR);
            return;
        }
        Set<String> types = collectPassengerTypes();
        System.out.println(SEPARATOR);
        System.out.println("RUNNING TOTALS:");
        for (String type : types) {
            BigDecimal total = getTotalChargedForType(type);
            BigDecimal cap   = fareCalculator.getDailyCap(type);
```

```
            System.out.println("  " + type + ": £" + total + " / £" + cap +
" cap");
        }
        System.out.println(SEPARATOR);
    }

    public void showSummary() {
        if (journeys.isEmpty()) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys to summarise.");
            System.out.println(SEPARATOR);
            return;
        }
        int[]        counts  = countTimeBands();
        BigDecimal[] totals  = calculateTotals();
        int          maxId   = findMostExpensiveId();

        BigDecimal totalCharged = totals[0];
        BigDecimal totalUncapped = totals[1];
        BigDecimal average = totalCharged.divide(
                BigDecimal.valueOf(journeys.size()), 2,
RoundingMode.HALF_UP);
        BigDecimal savings = totalUncapped.subtract(totalCharged);

        printSummary(totalCharged, average, maxId, savings, counts[0],
counts[1]);
    }

    public boolean hasJourneys() {
        return !journeys.isEmpty();
    }

    public List<Journey> getJourneys() {
        return Collections.unmodifiableList(journeys);
    }

    private BigDecimal getTotalChargedForType(String passengerType) {
        BigDecimal total = BigDecimal.ZERO;
        for (Journey journey : journeys) {
            if (journey.getPassengerType().equalsIgnoreCase(passengerType))
{
                total = total.add(journey.getChargedFare());
            }
        }
        return total;
    }

    private int findIndexById(int id) {
        for (int i = 0; i < journeys.size(); i++) {
            if (journeys.get(i).getId() == id) {
                return i;
            }
        }
        return -1;
    }
    # Oracle Corporation (n.d.)
    private Set<String> collectPassengerTypes() {
```

```
        Set<String> types = new LinkedHashSet<>();
        for (Journey j : journeys) {
            types.add(j.getPassengerType());
        }
        return types;
    }

    private int[] countTimeBands() {
        int peak = 0;
        int offPeak = 0;
        for (Journey j : journeys) {
            if (j.getTimeBand().equalsIgnoreCase("PEAK")) {
                peak++;
            } else {
                offPeak++;
            }
        }
        return new int[]{peak, offPeak};
    }
    # Oracle Corporation (n.d.)
    private BigDecimal[] calculateTotals() {
        BigDecimal totalCharged  = BigDecimal.ZERO;
        BigDecimal totalUncapped = BigDecimal.ZERO;
        for (Journey j : journeys) {
            totalCharged  = totalCharged.add(j.getChargedFare());
            totalUncapped = totalUncapped.add(
                    j.getBaseFare().subtract(j.getDiscountApplied()));
        }
        return new BigDecimal[]{totalCharged, totalUncapped};
    }

    private int findMostExpensiveId() {
        int maxId = -1;
        BigDecimal maxFare = BigDecimal.ZERO;
        for (Journey j : journeys) {
            if (j.getChargedFare().compareTo(maxFare) > 0) {
                maxFare = j.getChargedFare();
                maxId   = j.getId();
            }
        }
        return maxId;
    }

    private void printAddConfirmation(BigDecimal[] fareResult) {
        System.out.println(SEPARATOR);
        System.out.println("Journey added successfully.");
        System.out.println("Base fare    : £" + fareResult[0]);
        System.out.println("Discount     : £" + fareResult[1]);
        System.out.println("Charged fare : £" + fareResult[2]);
        System.out.println(SEPARATOR);
    }

    private void printSummary(BigDecimal total, BigDecimal average,
                              int maxId, BigDecimal savings,
                              int peakCount, int offPeakCount) {
        System.out.println(WIDE_SEP);
        System.out.println("  END OF DAY SUMMARY");
```

```
        System.out.println(WIDE_SEP);
        System.out.println("Total journeys         : " + journeys.size());
        System.out.println("Total cost             : £" + total);
        System.out.println("Average cost           : £" + average);
        System.out.println("Most expensive journey : ID " + maxId);
        System.out.println("Savings from cap       : £" + savings);
        System.out.println("Peak journeys          : " + peakCount);
        System.out.println("Off-peak journeys      : " + offPeakCount);
        System.out.println(WIDE_SEP);
    }
}
```

RiderProfile.java

```
// RiderProfile Class
public class RiderProfile {

    private String name;
    private String passengerType;
    private String defaultPayment;

    public RiderProfile(String name, String passengerType, String
defaultPayment) {
        this.name = name;
        this.passengerType = passengerType;
        this.defaultPayment = defaultPayment;
    }

    public String getName() {
        return name;
    }

    public String getPassengerType() {
        return passengerType;
    }

    public String getDefaultPayment() {
        return defaultPayment;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setPassengerType(String passengerType) {
        this.passengerType = passengerType;
    }

    public void setDefaultPayment(String defaultPayment) {
        this.defaultPayment = defaultPayment;
    }

    public void displayProfile() {
        System.out.println("--------------------");
        System.out.println("Name           : " + name);
        System.out.println("Passenger Type : " + passengerType);
```

```
        System.out.println("Default Payment: " + defaultPayment);
        System.out.println("--------------------");
    }
}
```

RiderSession.java

```
import java.util.List;
import java.util.Scanner;

// RiderSession Class
public class RiderSession {

    private static final String SEPARATOR    = "--------------------";
    private static final String WIDE_SEP     = "====================";

    private final Scanner scanner;
    private final ConfigManager configManager;
    private final FareCalculator fareCalculator;
    private final JourneyManager journeyManager;
    private final FileManager fileManager;
    private RiderProfile profile;

    public RiderSession(Scanner scanner, ConfigManager configManager) {
        this.scanner       = scanner;
        this.configManager = configManager;
        this.fareCalculator = new FareCalculator(configManager);
        this.journeyManager = new JourneyManager(fareCalculator);
        this.fileManager   = new FileManager();
    }

    public void start() {
        setupProfile();
        runRiderMenu();
    }

    private void setupProfile() {
        System.out.println(SEPARATOR);
        System.out.println("1. Load existing profile");
        System.out.println("2. Create new profile");
        System.out.println(SEPARATOR);
        System.out.print("Choose: ");
        String choice = scanner.nextLine().trim();

        if (choice.equals("1")) {
            loadExistingProfile();
        } else {
            createNewProfile();
        }
    }

    private void loadExistingProfile() {
        System.out.print("Enter profile filename (e.g.
deborah_profile.json): ");
        String filePath = scanner.nextLine().trim();
        profile = fileManager.loadProfile(filePath);
```

```
        if (profile == null) {
            System.out.println("Could not load profile. Creating a new one
instead.");
            createNewProfile();
        }
    }

    private void createNewProfile() {
        System.out.println(SEPARATOR);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine().trim();

        String passengerType = promptPassengerType();

        System.out.print("Default payment option (e.g. CARD, CASH): ");
        String payment = scanner.nextLine().trim().toUpperCase();

        profile = new RiderProfile(name, passengerType, payment);
        System.out.println("Profile created. Welcome, " + name + "!");
    }

    private void runRiderMenu() {
        boolean running = true;
        while (running) {
            showRiderMenu();
            String choice = scanner.nextLine().trim();
            running = handleRiderChoice(choice);
        }
    }

    private void showRiderMenu() {
        System.out.println("\n" + WIDE_SEP);
        System.out.println("  CITYRIDE LITE - " + profile.getName());
        System.out.println(WIDE_SEP);
        System.out.println("1.  View profile");
        System.out.println("2.  Add journey");
        System.out.println("3.  View all journeys");
        System.out.println("4.  Filter journeys by type");
        System.out.println("5.  Remove journey");
        System.out.println("6.  View running totals");
        System.out.println("7.  End-of-day summary");
        System.out.println("8.  Export journeys to CSV");
        System.out.println("9.  Export summary (CSV + text)");
        System.out.println("10. Import journeys from CSV");
        System.out.println("11. Save profile");
        System.out.println("12. Save and exit");
        System.out.print("Enter choice: ");
    }

    private boolean handleRiderChoice(String choice) {
        switch (choice) {
            case "1":  profile.displayProfile();           return true;
            case "2":  handleAddJourney();                 return true;
            case "3":  journeyManager.listJourneys();      return true;
            case "4":  handleFilterJourneys();             return true;
            case "5":  handleRemoveJourney();              return true;
            case "6":  journeyManager.showRunningTotals(); return true;
```

```
            case "7":  journeyManager.showSummary();       return true;
            case "8":  handleExportJourneys();             return true;
            case "9":  handleExportSummary();              return true;
            case "10": handleImportJourneys();             return true;
            case "11": handleSaveProfile();                return true;
            case "12": handleSaveAndExit();                return false;
            default:
                System.out.println("Invalid option. Please enter 1-12.");
                return true;
        }
    }

    private void handleAddJourney() {
        showFareReference();

        System.out.print("Passenger name (Enter to use profile name): ");
        String name = scanner.nextLine().trim();
        if (name.isEmpty()) {
            name = profile.getName();
        }

        int fromZone    = promptZone("from");
        int toZone      = promptZone("to");
        String timeBand = promptTimeBand();

        System.out.print("Passenger type [" + profile.getPassengerType()
                + "] (Enter to use profile): ");
        String typeInput     = scanner.nextLine().trim().toUpperCase();
        String passengerType = typeInput.isEmpty() ?
profile.getPassengerType() : typeInput;

        journeyManager.addJourney(name, fromZone, toZone, timeBand,
passengerType);
    }

    private void showFareReference() {
        System.out.println(SEPARATOR);
        System.out.println("FARE REFERENCE TABLE");
        System.out.println(SEPARATOR);
        System.out.println("Zone Pair | PEAK   | OFF_PEAK");
        System.out.println("----------+--------+---------");
        int[] zones = {1, 2, 3, 4, 5};
        for (int from : zones) {
            for (int to : zones) {
                String peak    = "£" + configManager.getBaseFare(from, to,
"PEAK");
                String offPeak = "£" + configManager.getBaseFare(from, to,
"OFF_PEAK");
                System.out.printf("  %d  →  %d  | %-6s | %s%n", from, to,
peak, offPeak);
            }
        }
        System.out.println(SEPARATOR);
        System.out.println("DISCOUNTS:");
        String[] types = {"ADULT", "STUDENT", "CHILD", "SENIOR_CITIZEN"};
        for (String t : types) {
            java.math.BigDecimal rate = configManager.getDiscountRate(t);
```

```
            java.math.BigDecimal cap  = configManager.getDailyCap(t);
            System.out.printf("  %-15s discount: %-4s  daily cap: £%s%n",
                    t,
                    rate.multiply(new
java.math.BigDecimal("100")).stripTrailingZeros().toPlainString() + "%",
                    cap);
        }
        System.out.println(SEPARATOR);
    }

    private void handleFilterJourneys() {
        String type = promptPassengerType();
        journeyManager.filterByPassengerType(type);
    }

    private void handleRemoveJourney() {
        if (!journeyManager.hasJourneys()) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys to remove.");
            System.out.println(SEPARATOR);
            return;
        }
        journeyManager.listJourneys();
        System.out.print("Enter journey ID to remove: ");
        int id = readInt();
        journeyManager.removeJourney(id);
    }

    private void handleExportJourneys() {
        if (!journeyManager.hasJourneys()) {
            System.out.println("No journeys to export.");
            return;
        }
        fileManager.exportJourneysCsv(journeyManager.getJourneys(),
profile.getName());
    }

    private void handleExportSummary() {
        if (!journeyManager.hasJourneys()) {
            System.out.println("No journeys to export.");
            return;
        }
        journeyManager.showSummary();
        fileManager.exportSummaryCsv(journeyManager.getJourneys(),
profile.getName());
        fileManager.exportSummaryText(buildSummaryText(),
profile.getName());
    }

    private void handleImportJourneys() {
        System.out.print("Enter CSV filename to import (e.g. journeys.csv):
");
        String filePath = scanner.nextLine().trim();
        List<String[]> rows = fileManager.importJourneysCsv(filePath);

        int added = 0;
        for (String[] row : rows) {
```

```
            try {

                int from     = Integer.parseInt(row[3].trim());
                int to       = Integer.parseInt(row[4].trim());
                String band  = row[5].trim();
                String type  = row[6].trim();
                String name  = row[2].trim();
                journeyManager.addJourney(name, from, to, band, type);
                added++;
            } catch (Exception e) {

            }
        }
        System.out.println(added + " journey(s) added from import.");
    }

    private void handleSaveProfile() {
        String filePath = profile.getName().replaceAll("\\s+",
"_").toLowerCase()
                + "_profile.json";
        fileManager.saveProfile(profile, filePath);
    }

    private void handleSaveAndExit() {
        System.out.print("Save profile and journeys before exiting? (Y/N):
");
        String confirm = scanner.nextLine().trim().toUpperCase();
        if (confirm.equals("Y")) {
            handleSaveProfile();
            handleExportJourneys();
        }
        System.out.println(SEPARATOR);
        System.out.println("Session ended. Goodbye, " + profile.getName() +
"!");
        System.out.println(SEPARATOR);
    }

    private int promptZone(String label) {
        int zone = 0;
        boolean valid = false;
        while (!valid) {
            System.out.print("Enter " + label + " zone (1-5): ");
            zone = readInt();
            if (zone >= 1 && zone <= 5) {
                valid = true;
            } else {
                System.out.println("Zone must be between 1 and 5.");
            }
        }
        return zone;
    }

    private String promptTimeBand() {
        String band = "";
        boolean valid = false;
        while (!valid) {
            System.out.print("Time band (PEAK / OFF_PEAK): ");
```

```
            band = scanner.nextLine().trim().toUpperCase();
            if (band.equals("PEAK") || band.equals("OFF_PEAK")) {
                valid = true;
            } else {
                System.out.println("Please enter PEAK or OFF_PEAK.");
            }
        }
        return band;
    }

    private String promptPassengerType() {
        String type = "";
        boolean valid = false;
        while (!valid) {
            System.out.println("Passenger types: ADULT | STUDENT | CHILD |
SENIOR_CITIZEN");
            System.out.print("Enter type: ");
            type = scanner.nextLine().trim().toUpperCase();
            if (type.equals("ADULT") || type.equals("STUDENT")
                    || type.equals("CHILD") ||
type.equals("SENIOR_CITIZEN")) {
                valid = true;
            } else {
                System.out.println("Invalid type. Please try again.");
            }
        }
        return type;
    }

    private int readInt() {
        int value = 0;
        boolean valid = false;
        while (!valid) {
            try {
                value = Integer.parseInt(scanner.nextLine().trim());
                valid = true;
            } catch (NumberFormatException e) {
                System.out.print("Please enter a whole number: ");
            }
        }
        return value;
    }

    private String buildSummaryText() {
        return "CityRide Lite - End of Day Summary\n"
                + "Rider: " + profile.getName() + "\n"
                + "Date : " + java.time.LocalDate.now() + "\n"
                + "See CSV file for full journey breakdown.\n";
    }
}
```

References

GeeksforGeeks (2025). *BigDecimal Class in Java.* Retrieved from: https://www.geeksforgeeks.org/bigdecimal-class-java/ [Accessed 3 April 2026].

GeeksforGeeks (2026, February). *ArrayList in Java.* Retrieved from: https://www.geeksforgeeks.org/arraylist-in-java/ [Accessed 2 April 2026].

GeeksforGeeks (2026, January). *HashMap in Java*. Retrieved from: https://www.geeksforgeeks.org/java-util-hashmap-in-java-with-examples/ [Accessed 2 April 2026].

help4code (n.d.). *bus-booking-system.* Retrieved from: https://github.com/help4code/bus-booking-system [Accessed 20 March 2026].

Oracle Corporation (n.d.). *Class BigDecimal (Java Platform SE 8).* Retrieved from: https://docs.oracle.com/javase/8/docs/api/java/math/BigDecimal.html [Accessed 2 April 2026].

Oracle Corporation (n.d.). *Class LinkedHashSet (Java Platform SE 8).* Retrieved from: https://docs.oracle.com/javase/8/docs/api/java/util/LinkedHashSet.html [Accessed 4 April 2026].

Oracle Corporation (n.d.). *Class LocalDate (Java Platform SE 8).* Retrieved from: https://docs.oracle.com/javase/8/docs/api/java/time/LocalDate.html [Accessed 4 April 2026].

Oracle Corporation (n.d.). *Class Scanner (Java Platform SE 8).* Retrieved from: https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html [Accessed 1 April 2026].

Oracle Corporation (n.d.). *Class String (Java Platform SE 8).* Retrieved from: https://docs.oracle.com/javase/8/docs/api/java/lang/String.html [Accessed 4 April 2026].

Oracle Corporation (n.d.). *Reading, Writing, and Creating Files.* Retrieved from: https://docs.oracle.com/javase/tutorial/essential/io/file.html [Accessed 29 March 2026].

Oracle Corporation (n.d.). *The Java Tutorials - Exceptions*. Retrieved from: https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html [Accessed 3 April 2026].

Appendices

Milestone 1 Part 2

Purpose of the Program

The aim of this program is to develop a console - based Java application named CityRide Lite, designed to record and manage public transport journeys within a single day. The system enables users to enter key journey details such as passenger name, travel zones, passenger category, and time band. Using a predefined fare structure that includes base fares, discounts, and daily caps, the application calculates the total cost of each journey accurately. CityRide Lite Part 2 expands on the initial version developed in Part 1. While the earlier version operated as a single - session, in memory system, this updated version introduces a more advanced and persistent transport management solution. It incorporates multiple user roles and allows data to be stored between sessions. Users can now create

and manage profiles, record journeys over time, and rely on configurable transport rules to calculate fares, making the system more flexible and realistic.

To improve overall functionality and user experience, several enhancements have been added. These include the ability to filter journeys based on different criteria, calculate totals according to passenger type, reset journey records for a new day, and enforce stronger input validation. The system also generates more detailed outputs, such as journey summaries and the number of zones travelled, providing clearer insights for users. Furthermore, the application supports two main roles: Rider and Admin. Riders can manage personal profiles, add and organise journeys, import and export data using CSV files, and generate end of day reports. All relevant data is stored persistently using JSON and CSV formats. The Admin role, secured with a password, allows authorised users to modify fare settings such as base fares, discounts, daily caps, and peak travel periods, with validation checks to ensure accuracy. The original fare calculation method remains in place, factoring in zones travelled and time bands, while applying discounts and caps automatically. Overall, the program demonstrates advanced concepts such as file handling, data validation, role - based access control, and dynamic recalculation within a structured and user friendly console interface.

Core Program Functionality

Role Selection: On launch, the user chooses between the Rider and Admin roles. This determines which menu and features are accessible. If Rider is selected; Load the RiderMenu, If Admin is selected; Prompt for a password. For Example, user enters "Admin". System asks: "Enter Admin Password:". User enters "CityRide2026". If correct, access is granted; otherwise, return to the main role selection.

Configuration Loading: The system loads its fare rules, discounts, daily caps, and peak time windows from a JSON configuration file (e.g config.json) file at startup. If the file is missing, safe default values are used automatically.

Rider Profile Management: A Rider can create a new profile, save it to a JSON file, or load a previously saved profile. The profile stores the Rider's name, passenger type, and default payment option. A user named "Deborah" saves her profile. Next time she uses the app, she selects "Load Profile," and the system automatically sets the Passenger Type to "Student" without her having to type it again.

Journey Management: Riders can add journeys manually (entering zones, time band, and passenger type), edit existing journeys, or delete journeys with automatic recalculation of totals and caps. Journeys can also be imported in bulk from a CSV file e.g weekly_commute.csv.

Fare Calculation: The system calculates fares using the same dataset rules as Part 1: base fare by zone pair and time band, passenger discount applied, and daily cap enforced per passenger type.

End of Day Summary: The system produces a detailed summary that includes total journeys, total cost, average cost per journey, the most expensive journey (by ID), whether the daily cap was reached, savings compared to uncapped fares, and category counts (peak vs off - peak, counts per zone pair).

Export: The summary and journey list can be exported as both a CSV report (line items) and a human - readable text file. File names include the date and rider name for clear identification.

Admin Menu: Password protected access allows an Admin to view the active configuration and add, update, or delete fare rules, discounts, daily caps, and peak time windows. All input is validated before saving.

Save on Exit: When the Rider exits, the system offers to save the current day's profile and *journeys to JSON so they can be continued in a future session.*

In summary;

Rider Features:

- Create, load, and save rider profile (JSON)
- Add, edit, delete journeys
- Import journeys (CSV)
- Export journeys and reports (CSV + text)
- View running totals and summaries
- End of day report generation

Admin Features:

- Login with password
- View system configuration
- Add/update/delete (Base fares, Discounts, Daily caps & Peak time windows)
- Validate all inputs before saving

System Features:

- Load config file at startup
- Use default values if file missing
- Apply fare rules (Base fare, Discounts & Daily caps)
- Store data using (JSON (profiles/config) & CSV (journeys/reports))

System Constraints

- **Two roles only**: The system supports exactly two roles; Rider and Admin. Role is selected at startup.
- **Console - based interface**: All interaction is through a text menu driven console. No graphical interface is provided.
- **File - based persistence**: Profiles and configuration are stored as JSON; journeys and reports are stored as CSV. No database is used.

- **Config file dependency**: If the configuration JSON file is absent, the system defaults to safe built-in values rather than crashing.
- **Fixed passenger types:** Valid types are ADULT, STUDENT, CHILD, and SENIOR CITIZEN (as defined in CityRideDataset).
- **Fixed time bands**: Journeys must be either PEAK or OFF_PEAK.
- **Zone range 1 - 5**: From and to zones must be integers between 1 and 5 inclusive.
- **Daily caps per passenger type:** Each passenger type has its own cap; once reached, further journeys cost £0.
- **Admin password required:** The admin menu cannot be accessed without the correct password.
- **All admin changes validated**: No update to the config is saved unless it passes validation.
- **Dynamic recalculation on delete**: Deleting or undoing a journey triggers a full recalculation of totals and caps.
- **Unique journey IDs**: Each journey is assigned a unique ID for the session, used when editing or deleting.

Input Process Output Table

The IPO table identifies, for each main function, what data the user provides (Input), what the system does with it (Process), and what is produced (Output). Each feature is user initiated, ensuring the system remains user driven.

Feature/Task	Inputs	Processing	Outputs
Start Program	None (program launch)	Load config from JSON; if missing, apply safe defaults; initialise data structures	Config loaded; role selection menu shown
Select Role	Role choice: RIDER or ADMIN (String)	Validate input; route to Rider session or Admin login	Appropriate menu displayed
Admin Login	Password (String)	Compare input against stored password; grant or deny access; count attempts	Admin menu shown or access denied message
Load Config	Config file (JSON)	Parse JSON; populate base fares, discounts, caps, peak windows in ConfigManager	System configured; confirmation message
Create Rider Profile	Name (String), passenger type	Validate inputs; create RiderProfile object	Profile created in memory

Feature/Task	Inputs	Processing	Outputs
	(String), default payment (String)		
Save Rider Profile	Confirmation to save (Boolean)	Serialise RiderProfile to JSON; write to file	JSON profile file saved
Load Rider Profile	Profile filename or name (String)	Read JSON file; parse into RiderProfile object	Profile loaded into session
Add Journey	From zone (int), to zone (int), time band (String), passenger type (String), date/time (String)	Validate all inputs; calculate zones crossed; look up base fare; apply discount; check and apply daily cap; assign unique ID; store journey	Journey stored; fare and cap status displayed
Edit Journey	Journey ID (int), field to edit (String), new value (String/int)	Validate ID exists; validate new value; update journey; recalculate all totals and caps	Journey updated; updated totals displayed
Delete Journey	Journey ID (int)	Validate ID exists; remove journey; recalculate all totals and caps	Confirmation message; updated totals displayed
Import Journeys from CSV	CSV filename (String)	Read CSV file; parse each row; validate each journey; calculate fares; add to journey list	Journeys imported; count of imported/rejected journeys shown
Export Journeys to CSV	Confirmation to export (Boolean)	Format all journeys as CSV rows; write to file with date and rider name	CSV file saved; file path shown
View Journey List	View request (Boolean)	Loop through all stored journeys; format each for display	All journeys displayed with IDs, zones, fare, cap status
Filter Journeys by Type	Passenger type (String)	Filter journey list by matching passenger type; format for display	Matching journeys displayed
Calculate Fare	From zone (int), to zone (int), time band (String), passenger type (String)	Calculate zones crossed; look up base fare from dataset; apply passenger discount; check daily cap; return final charged fare	Final fare displayed; cap notification if relevant
View Running Totals	View request (Boolean)	Sum all journey fares for current session; compare to cap	Total cost, cap reached status, savings displayed

Feature/Task	Inputs	Processing	Outputs
End-of-Day Summary	Summary request (Boolean)	Count journeys; sum total/average cost; find most expensive journey ID; calculate savings vs uncapped fares; count peak/off-peak; count per zone pair	Detailed summary displayed on screen
Export Summary to CSV	Confirmation (Boolean)	Format summary data as CSV with line items; include rider name and date in filename	CSV report file saved
Export Summary to Text	Confirmation (Boolean)	Format summary as human readable text; include rider name and date in filename	Text summary file saved
Admin View Config	View request (Boolean)	Read active configuration from memory	All fare rules, discounts, caps, and peak windows displayed
Admin Add/Update Fare Rule	From zone (int), to zone (int), time band (String), new fare (BigDecimal)	Validate all fields; check fare is positive; update config in memory	Confirmation or detailed validation error message
Admin Update Discount	Passenger type (String), new discount rate (BigDecimal)	Validate passenger type; validate discount is between 0.00 and 1.00; update config	Confirmation or validation error
Admin Update Daily Cap	Passenger type (String), new cap (BigDecimal)	Validate passenger type; validate cap is positive; update config	Confirmation or validation error
Save Config	Confirmation to save (Boolean)	Save config to JSON; write to file	JSON config file saved; confirmation shown
Save and Exit	Exit confirmation (Boolean)	Offer to save profile and journeys; save to JSON if confirmed; close scanner	Farewell message; files saved if chosen

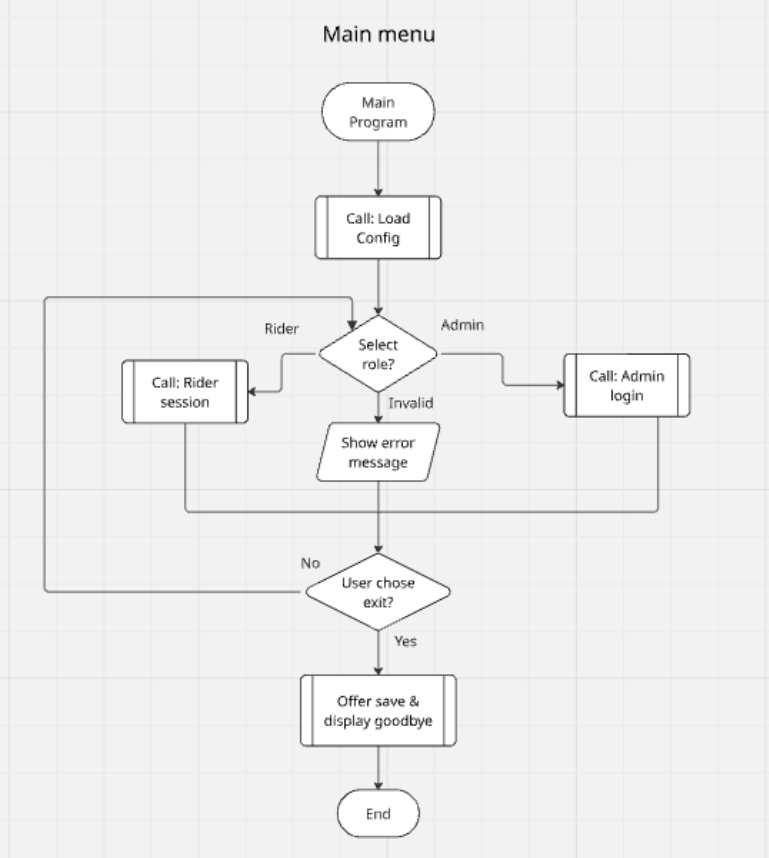
Algorithm Design

The algorithm design for CityRide Lite Part 2 uses flowcharts to represent each major subroutine. The flowcharts follow standard notation: rounded rectangles for start/end terminals, rectangles for processes, parallelograms for input/output operations, and diamonds for decisions. Subroutines are shown with their own named terminal at the top,

and flow returns to the calling routine via a Return terminal at the bottom. This section describes the logic of each algorithm in prose form.

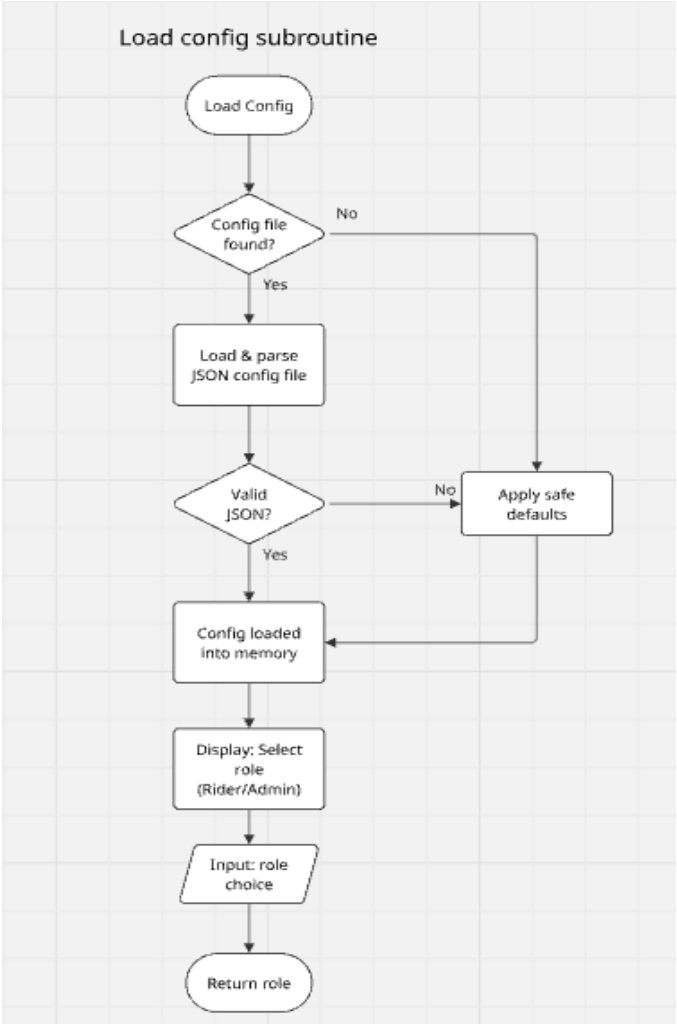
Main Menu

On program launch, the system calls the Load Config subroutine to read the JSON configuration file. If the file is found and valid, the config is loaded into memory. If it is missing or corrupted, safe default values are applied automatically. The system then displays the role selection menu. The user enters RIDER or ADMIN. If RIDER is selected, the Rider Session subroutine is called. If ADMIN is selected, the Admin Login subroutine is called. Any other input shows an error and re - displays the menu. The loop continues until the user enters EXIT.



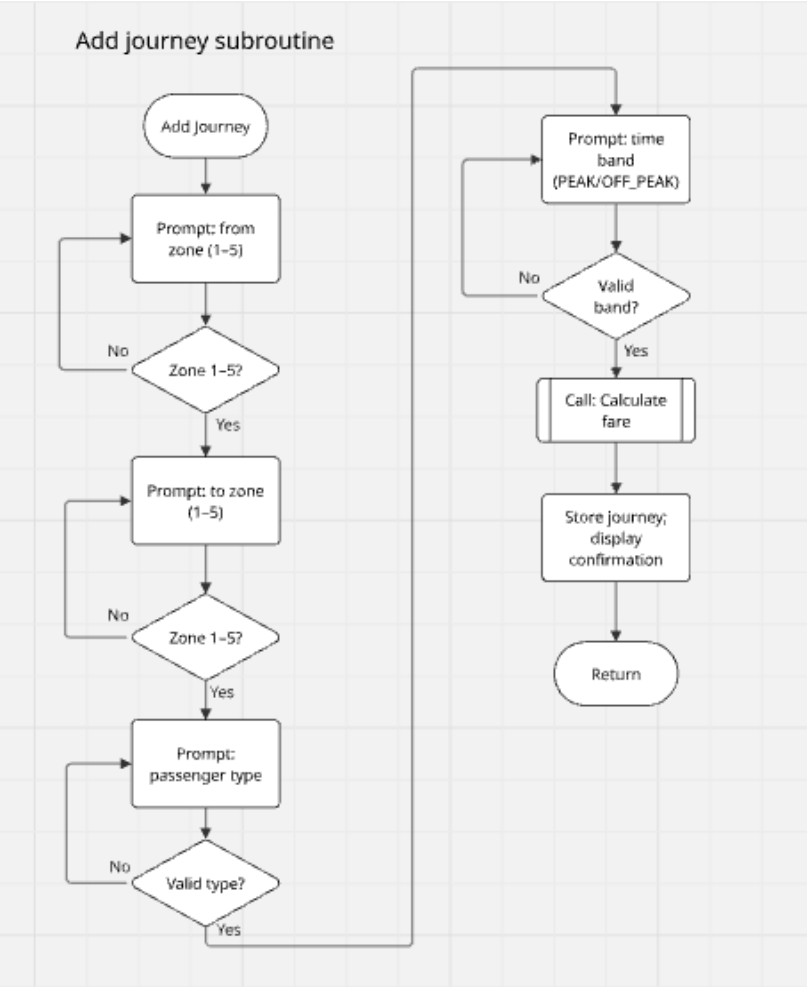
Load Config Subroutine

The subroutine checks whether the config JSON file exists. If found, it loads and parses the file into the ConfigManager, populating the base fare, discount rate, and daily cap maps. If the file is not found or is invalid, the system applies built-in default values and continues without crashing. A confirmation or warning message is displayed. The subroutine then returns to the caller.



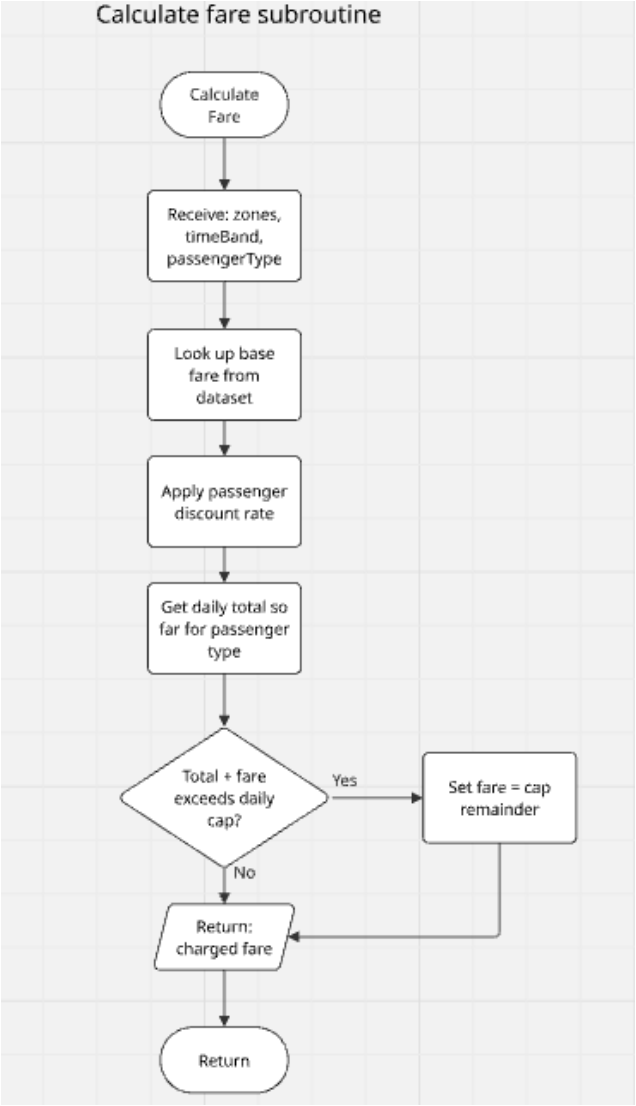
Add Journey Subroutine

The user is prompted for: from zone (validated as integer 1-5), to zone (validated as integer 1-5), time band (validated as PEAK or OFF_PEAK), and passenger type (validated against the four allowed types). Each input is validated in a loop that re-prompts on invalid input. Once all inputs are valid, the Calculate Fare subroutine is called. A Journey object is created and stored. A confirmation message showing the base fare, discount, and charged fare is displayed, and the subroutine returns.



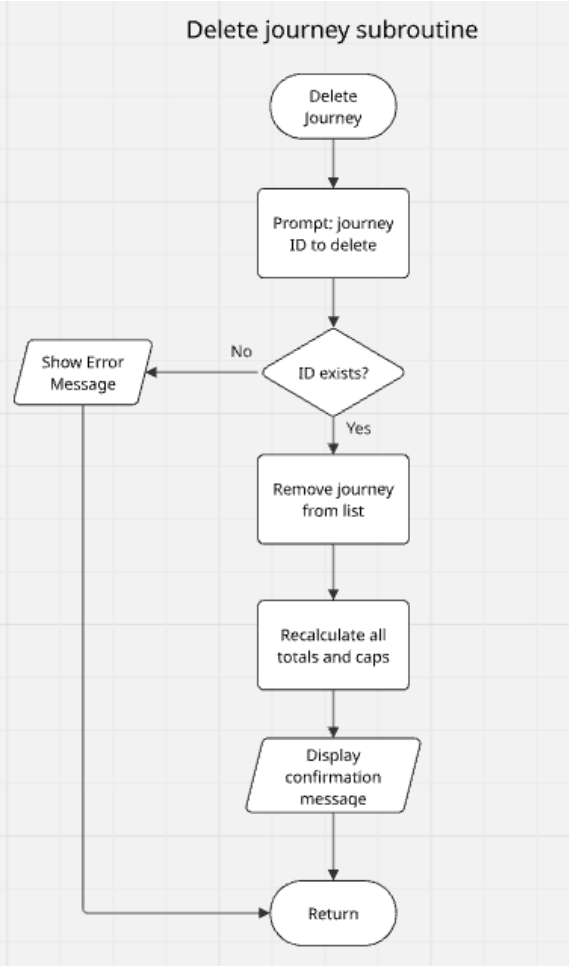
Calculate Fare Subroutine

The subroutine receives from zone, to zone, time band, and passenger type as parameters. It looks up the base fare from the ConfigManager using the zone pair and time band key. It then retrieves the discount rate for the passenger type and calculates the discount amount. The fare after discount is computed. The subroutine then retrieves the running total charged so far for this passenger type and checks whether adding the discounted fare would exceed the daily cap. If it would exceed the cap, the charged fare is reduced to the remaining cap allowance. If the running total already equals or exceeds the cap, the charged fare is set to zero. The final charged fare is returned alongside the base fare and discount applied.



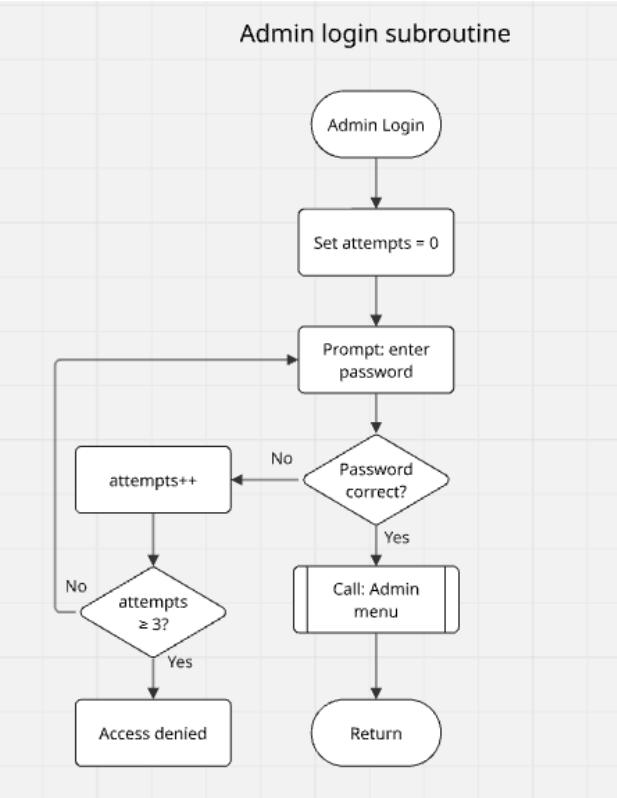
Delete Journey Subroutine

The user is prompted for a journey ID. If the ID does not exist in the journey list, an error message is displayed and the subroutine returns without making any changes. If the ID is found, the journey is removed from the list. The recalculate totals process then iterates through all remaining journeys in order, recalculating each fare from scratch using running totals to ensure daily cap logic is correctly reapplied. A confirmation message is displayed and the subroutine returns.



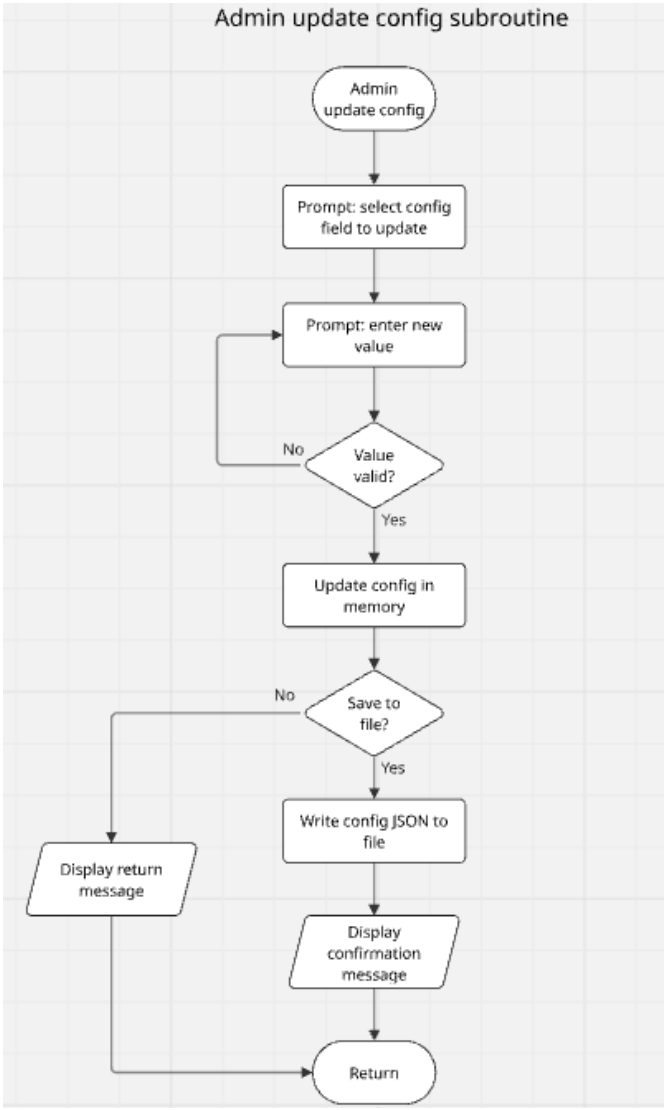
Admin Login Subroutine

The attempt counter is set to zero. The user is prompted to enter the admin password. If the password is correct, the Admin Menu subroutine is called and control returns from it. If incorrect, the attempt counter is incremented. If the counter reaches three, access is denied and the subroutine returns to the role selection menu. Otherwise, the user is re-prompted for the password, showing the number of remaining attempts.



Admin Update Config Subroutine

The admin selects which configuration field to update: base fare, discount rate, daily cap, or peak window. The system prompts for the relevant input fields. All inputs are validated before any change is made. If any field fails validation, a clear error message is shown and no changes are saved. If all inputs pass validation, the in-memory configuration is updated. The admin is then offered the option to save the updated configuration to the JSON file. A confirmation or error message is displayed accordingly.



Import Journeys from CSV Subroutine

The user is prompted for a CSV filename. The system attempts to open the file. If the file is not found, an error message is shown and the subroutine returns. For each row in the file, the system validates the number of columns and each individual field. If a row is invalid, it is skipped and counted as rejected. If valid, the Calculate Fare subroutine is called and a journey object is created and added to the list. After all rows are processed, a summary of imported and rejected rows is displayed and the subroutine returns.



Class Structure

The class diagram for CityRide Lite Part 2 shows the object-oriented structure of the system. The main classes and their relationships are as follows:

CityRideLite - the main entry point. Handles role selection and routes to RiderSession or AdminSession. Associates with both session classes.

RiderSession - manages the Rider menu loop. Associates with JourneyManager, RiderProfile, FileManager, FareCalculator, and ConfigManager.

AdminSession - manages the Admin menu loop. Associates with ConfigManager.

RiderProfile - stores rider name, passenger type, and default payment option. Contains getters and setters.

JourneyManager - manages the list of Journey objects (ArrayList). Handles add, edit, delete, filter, recalculate totals, and summary calculations. Aggregates Journey (one-to-many).

Journey - stores all journey data including ID, date, zones, time band, passenger type, base fare, discount applied, and charged fare.

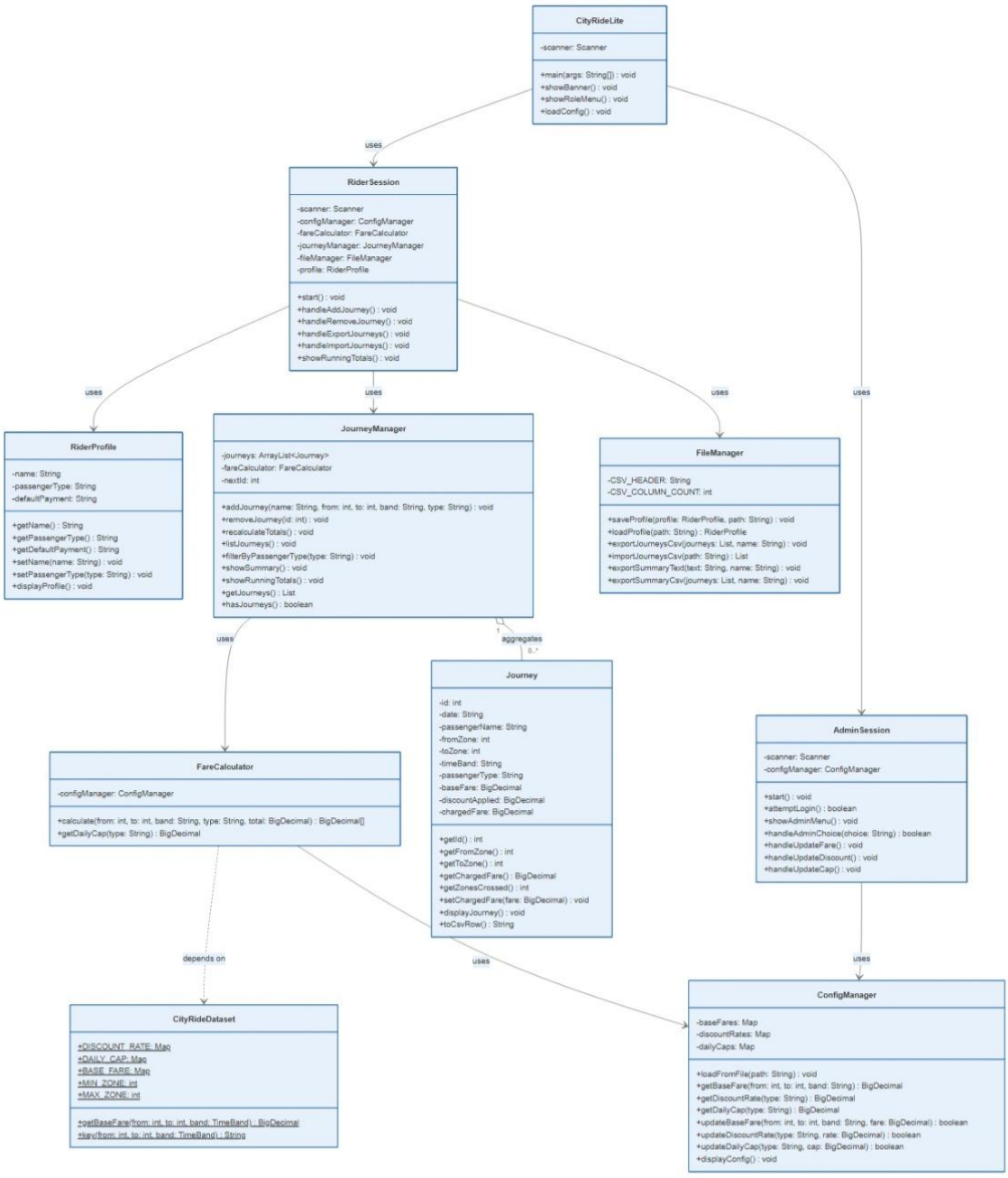
FareCalculator - calculates the charged fare given journey inputs, applying discount and daily cap logic. Associates with ConfigManager.

ConfigManager - loads, holds, and saves the system configuration using HashMaps for base fares, discounts, and caps. Reads/writes JSON.

FileManager - handles all file I/O: JSON read/write for profiles and config, CSV import/export for journeys, text and CSV export for summaries.

CityRideDataset - unchanged from Part 1. A static constants class providing the base fare data structure.

Relationships used: solid line with open arrowhead = Association (uses/calls); diamond with open arrowhead = Aggregation (has-a, can exist independently); no inheritance is used as no class extends another in this design.



Research (minimum of 1 required, preferrebly 2)

A terminal Python program that is structurally very close to CityRide Lite. Chosen specifically because it has three things directly comparable to Part 2: a PassengerDataCsv class that saves bookings to CSV (maps to FileManager), a separate Admin class with password authentication (maps to AdminSession), and fare calculation from combined parameters.

Name of program: Online Bus Ticket Reservation System - Console based Python Application

Reference (link): help4code (n.d.). *bus-booking-system.* Retrieved from: https://github.com/help4code/bus-booking-system [Accessed 20 March 2026].

What it does well (2-3 features that work effectively):

Persistent CSV storage for booking records - the program uses a dedicated PassengerDataCsv class whose sole responsibility is saving and loading passenger booking records to a CSV file. Each booking is written as a row with all key fields (name, departure, destination, fare, seat number, bus type). This means data survives between sessions, exactly the kind of file-based persistence required in CityRide Lite Part 2 for exporting journeys and summary reports. The separation into its own class also demonstrates single responsibility well.

Separate Admin class with its own menu - the program implements an Admin class that provides a distinct menu for managing bookings, separate from the passenger. The admin must authenticate before accessing this menu. This two-role structure (passenger user + admin) maps directly onto CityRide Lite Part 2's Rider and Admin role design.

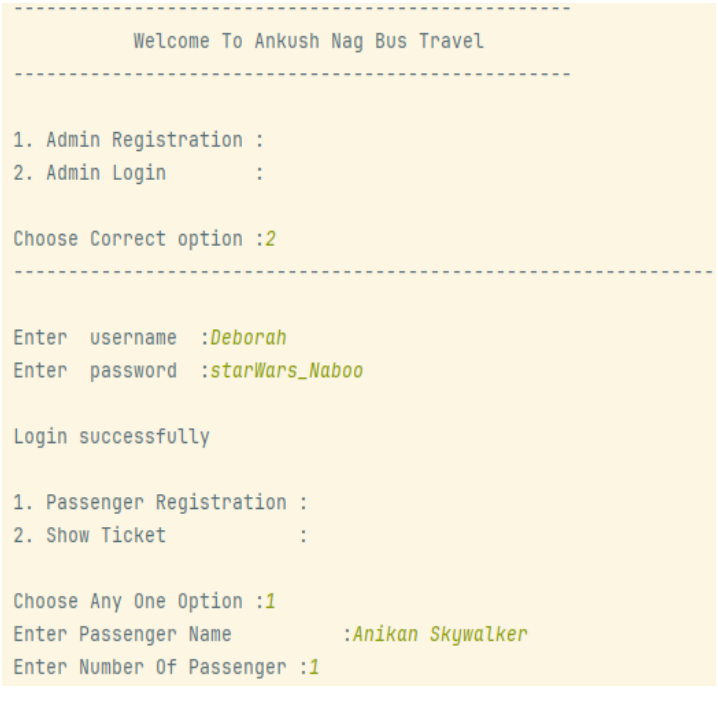
What it does poorly (at least 1 feature):

The program has no input validation - if the user enters an invalid bus type, seat number, or departure location, the program either crashes or silently accepts the bad input. There is also no daily cap concept, no discount system by passenger category, and no end of day summary or reporting feature. All booking data is stored in a single CSV with no ID lookup, meaning there is no way to remove or edit a specific booking by reference which is a limitation that CityRide Lite addresses through unique journey IDs and ID based deletion.

Key design ideas you could reuse (e.g., layout, navigation, input/output, program structure):

The most reusable idea is the dedicated CSV handler class pattern - a class whose only job is reading from and writing to a specific file. In CityRide Lite Part 2 this becomes the FileManager class, with separate methods for profile JSON, journey CSV, and summary export, keeping file I/O completely separate from the business logic in JourneyManager.

Screenshot (showing the interface/output):



```
Enter Number Of Passenger :1
1: Nagpur
2: Pune
3: Mumbai
4: Delhi
Enter Departure Location2
1: Gujrat
2: Raipur
3: Patna
4: Bhopal
Enter Destination Location  :3
Enter Date of Joiurney Like 07-05-1992   :20-03-2026
[1]__[2]__[3]__[4]__[5]__[6]__[7]__[8]__[9]__[10]
[11]_[12]_[13]_[14]_[15]_[16]_[17]_[18]_[19]_[20]
[21]_[22]_[23]_[24]_[25]_[26]_[27]_[28]_[29]_[30]
Choose a Seat Number & Max To Max You Can Book Two Ticket  :15
Do You Want To Book One More Seat Enter (Yes/No)
No
 1. AC BUS     = 500 Fare
 2. NON AC BUS = 300 Fare
Select Bus Type  :2

Number of Records Are Found In Database : 2
Data Save successfully
```

```
--------------------------------------------------
          Welcome To Ankush Nag Bus Travel
--------------------------------------------------

1. Admin Registration :
2. Admin Login       :

Choose Correct option :2
------------------------------------------------------------

Enter  username  :Deborah
Enter  password  :starWars_Naboo

Login successfully

1. Passenger Registration :
2. Show Ticket            :

Choose Any One Option :2
Enter Your Booking Id  :2
--------------------------------------------------------------------------------
                        Ankush Nag Bus Travel
--------------------------------------------------------------------------------
```

```
Choose Any One Option :2
Enter Your Booking Id  :2
--------------------------------------------------------------------------------
                         Ankush Nag Bus Travel
--------------------------------------------------------------------------------

 e_Ticket : Nagpur Address             : Hingna Road Priyadarshini T-Point
            Phone No\Mob No            : 8000800088,8888880000

 Pune ------------> Patna                    Passenger Id: 2

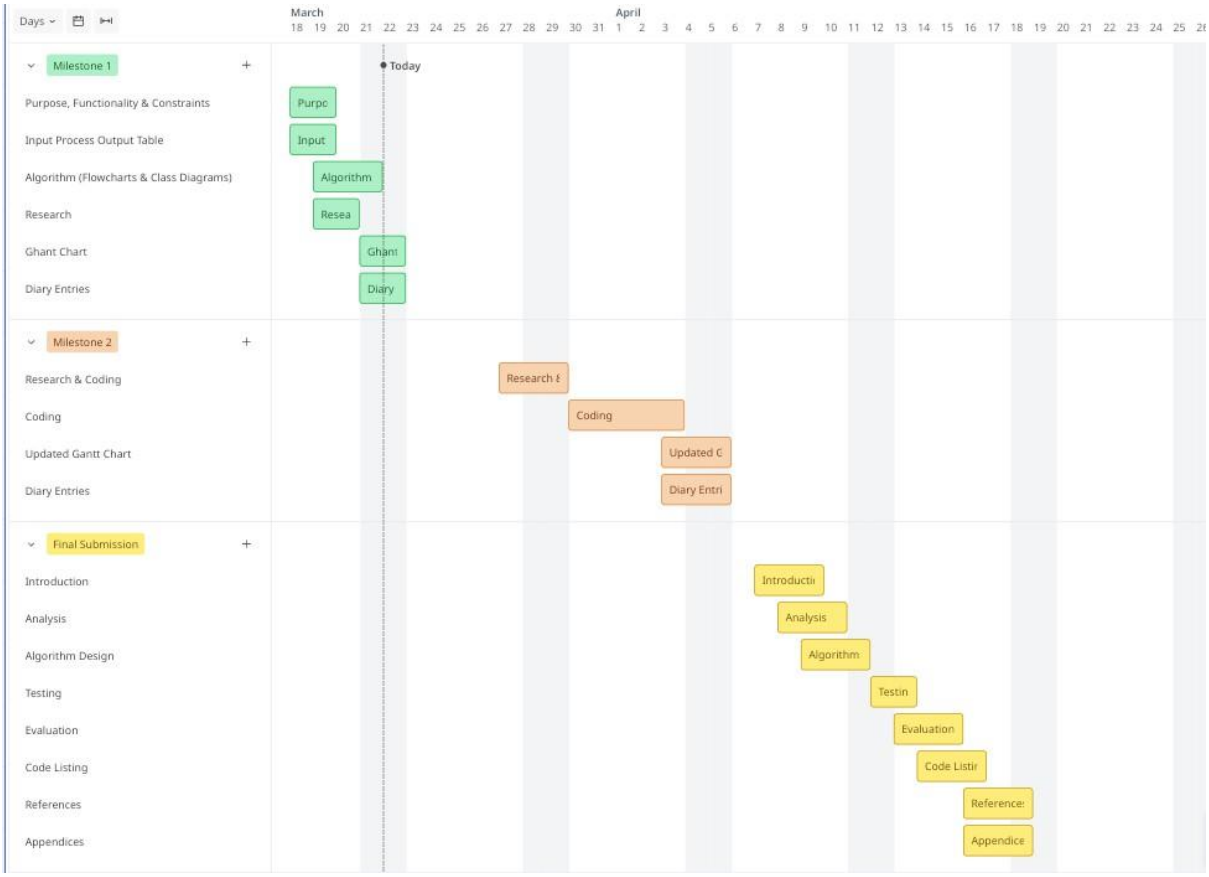
 Passenger Name : Anikan Skywalker              Number of Passenger : 1
--------------------------------------------------------------------------------

 Date of Booking : 20-03-2026               Seat No : [15]

 Bus Type :        NON AC BUS
 Bus Fare :        300

--------------------------------------------------------------------------------
```

Gantt Chart



Diary Entries

20/03/2026 Diary Entry 1: Analysis and Planning.

Today I began work on Part 2 of the CityRide Lite project. My first step was to carefully re read the Part 2 assignment brief in full, alongside the feedback from my Part 1 submission, so I could understand exactly what was expected and what needed to be improved. I then wrote the Purpose of the Program section, identifying what the extended system must do that is new compared to Part 1; specifically, the two role structure (Rider and Admin), persistent file storage using JSON and CSV, rider profile management, and the enhanced end of day summary with export functionality.

After that, I worked through all functional requirements listed in the brief and proceeded each one to in the IPO table. I was careful this time to only include genuine user inputs in the Input column, in my Part 1 submission, the tutor noted that some "inputs" were actually internal data structures rather than user provided values. I corrected this approach throughout the new IPO table, making sure each input column entry represents something the user actually types or provides.

I also noted the non-functional constraints carefully, which includes the console only interface, file based persistence, admin password protection, and the need for all admin changes to be validated before saving. I listed these under the System Constraints section as they are important for understanding the boundaries of the program.

22/03/2026 – Diary Entry 2: Algorithm, Research and Ghant Chart.

Today I focused developing the design and planning components of my Part 2 project. I began by creating flowcharts to represent the main algorithms of the system, including user interactions such as creating a rider profile, adding journeys, and handling admin configuration updates. While doing this, I paid close attention to using the correct flowchart symbols and improving the layout, as I received feedback in Part 1 about incorrect notation and poor readability. This time, i use corrected flowchart notation throughout using subroutine symbols for called functions, proper decision diamonds, and the subroutine name (not "Start") at the top of each sub flowchart. I ensured that all processes, decisions, and inputs/outputs were clearly represented and logically structured.

I also worked on designing the class diagram for the system. I identified key classes such as Rider, Journey, Admin, and Configuration, and defined their attributes and methods. I made sure to apply object - oriented principles such as encapsulation and separation of responsibilities, so that each class has a clear and specific role. One challenge I faced was deciding how different classes should interact, particularly how journeys are linked to rider profiles and how configuration data is accessed across the system. I resolved this by structuring relationships carefully and ensuring consistency in data flow.

In addition, I conducted research based on other programs, specifically working with JSON and CSV formats, as these are required for storing profiles, journeys, and reports. This research helped me understand how data persistence can be implemented and influenced my design decisions, particularly in separating file handling logic from core

program logic. Finally, I drew up the Gantt chart to plan the work across the available weeks, separating Milestone 1, Milestone 2 and the final submission into distinct weekly tasks to organise my project timeline, breaking the work such as analysis, design, implementation, testing, and final submission. This has helped me plan my time more effectively and ensure that I stay on track with deadlines.

Milestone 2 Part 2

Research (minimum of 2, at least 3)

Research 1

Title of research: Java File I/O - Reading and Writing Files Using BufferedReader and BufferedWriter (Oracle Documentation)

Reference (link): Oracle Corporation (n.d.). *Reading, Writing, and Creating Files*. Retrieved from: https://docs.oracle.com/javase/tutorial/essential/io/file.html [Accessed 29 March 2026].

How does the research help with coding practise?:

CityRide Lite Part 2 requires reading a JSON configuration file at startup and writing CSV files for journey exports and summary reports. This Oracle documentation explains how to use BufferedReader with FileReader to read files line by line, and BufferedWriter with FileWriter to write content to a file. It also explains how to check whether a file exists before attempting to open it using File.exists(), which is directly needed for the requirement that the program should fall back to safe default values if the configuration file is missing rather than crashing. The documentation also covers wrapping all file operations in try-catch blocks to handle IOException gracefully, which is essential for robust error management in the FileManager class.

Key coding ideas you could reuse in your program:

- Using BufferedReader and FileReader inside a try-catch block to open and read JSON and CSV files line by line
- Using File.exists() to check for the config file before attempting to load it, then defaulting to built-in constants if it is absent
- Using BufferedWriter and FileWriter to write CSV journey exports and text summary files
- Closing all streams using try-with-resources (e.g. try (BufferedReader br = new BufferedReader(...))) so resources are automatically released even if an exception occurs

Screenshot of research:

The `newBufferedReader(Path, Charset)` method opens a file for reading, returning a `BufferedReader` that can be used to read text from a file in an efficient manner.

The following code snippet shows how to use the `newBufferedReader` method to read from a file. The file is encoded in "US-ASCII."

```
Charset charset = Charset.forName("US-ASCII");
try (BufferedReader reader = Files.newBufferedReader(file, charset)) {
    String line = null;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException x) {
    System.err.format("IOException: %s%n", x);
}
```

Writing a File by Using Buffered Stream I/O

You can use the `newBufferedWriter(Path, Charset, OpenOption...)` method to write to a file using a `BufferedWriter`.

The following code snippet shows how to create a file encoded in "US-ASCII" using this method:

```
Charset charset = Charset.forName("US-ASCII");
String s = ...;
try (BufferedWriter writer = Files.newBufferedWriter(file, charset)) {
    writer.write(s, 0, s.length());
} catch (IOException x) {
    System.err.format("IOException: %s%n", x);
}
```

Research 2

Title of research: Working with ArrayList in Java (GeeksforGeeks)

Reference (link): GeeksforGeeks (2026, February). ArrayList in Java. Retrieved from: ArrayList in Java - GeeksforGeeks [Accessed 02 April 2026].

How does the research help with coding practise?:

In Part 1, journeys were stored in a fixed-size array (Journey[] journeys = new Journey[5]). This was flagged in the Part 1 feedback as a limitation — a fixed-size array cannot grow dynamically, meaning the program would fail if more than five journeys were added. For Part 2, the JourneyManager class must use ArrayList<Journey> instead, which grows automatically as journeys are added and supports removing journeys by index without manually shifting elements. This GeeksforGeeks article explains how to declare an ArrayList, add elements with .add(), remove elements with .remove(index), access elements with .get(index), and get the current size with .size(). It also explains how to iterate over an ArrayList using an enhanced for loop, which is cleaner and less error-prone than a counter-based loop with a manual size variable.

Key coding ideas you could reuse in your program:

- Replacing Journey[] journeys = new Journey[5] and int journeyCount with ArrayList<Journey> journeys = new ArrayList<>() in JourneyManager
- Using journeys.add(journey) instead of journeys[journeyCount] = journey; journeyCount++
- Using journeys.remove(i) instead of the manual shift loop used in Part 1's removeJourney() method

- Using journeys.size() instead of a separate journeyCount variable
- Iterating with for (Journey j : journeys) for list operations such as listing, filtering, and summary calculations.

Screenshot of research:

Hierarchy of ArrayList

It implements List Interface which is a sub-interface of Collection Interface.



Research 3

Title of research: Java HashMap - Storing and Retrieving Key-Value Pairs (GeeksforGeeks)

Reference (link): GeeksforGeeks (2026, January). *HashMap in Java*. Retrieved from: HashMap in Java - GeeksforGeeks [Accessed 02 April 2026].

How does the research help with coding practise?:

The ConfigManager class in Part 2 needs to store and retrieve configurable fare rules, passenger discounts, and daily caps. Rather than hardcoding these as constants (as in CityRideDataset in Part 1), Part 2 loads them from a JSON file into HashMap data structures so that admins can update them at runtime. This research explains how HashMap<Key, Value> stores key-value pairs, how to use .put(key, value) to add or update an entry, .get(key) to retrieve a value, .containsKey(key) to check whether a key exists before reading it, and .keySet() to iterate over all keys. Understanding these operations is essential for the ConfigManager, which must hold maps for base fares (Map<String, BigDecimal>), discounts (Map<PassengerType, BigDecimal>), and daily caps (Map<PassengerType, BigDecimal>), and allow admins to update individual entries through the admin menu.

Key coding ideas you could reuse in your program:

- Using HashMap<String, BigDecimal> in ConfigManager to store base fares keyed by a zone-and-timeband string (e.g. "1-3-PEAK")
- Using HashMap<PassengerType, BigDecimal> for discount rates and daily caps, keyed by passenger type enum

- Using .put(key, value) to update a config entry when the admin changes a fare rule
- Using .get(key) to retrieve a fare or discount in FareCalculator
- Using .containsKey(key) to validate that a fare rule exists before attempting to update it

Screenshot of research:

- K -> Type of keys maintained by this map
- V -> Type of mapped values

```
import java.util.HashMap;
import java.util.Map;

public class GFG{

    public static void main(String[] args){

        // Create a HashMap
        HashMap<String, Integer> hashMap = new HashMap<>();

        // Add elements to the HashMap
        hashMap.put("John", 25);
        hashMap.put("Jane", 30);
        hashMap.put("Jim", 35);

        // Iterate through the HashMap
        for (Map.Entry<String, Integer> entry : hashMap.entrySet()) {
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        }
    }
}
```

Program Code

AdminSession.java

```
import java.math.BigDecimal;
import java.util.Scanner;

// AdminSession Class
public class AdminSession {

    private static final String ADMIN_PASSWORD = "CityRide2026";
    private static final int MAX_ATTEMPTS = 3;
    private static final String SEPARATOR = "--------------------";

    private final Scanner scanner;
    private final ConfigManager configManager;

    public AdminSession(Scanner scanner, ConfigManager configManager) {
        this.scanner = scanner;
        this.configManager = configManager;
    }
```

```
    public void start() {
        boolean authenticated = attemptLogin();
        if (!authenticated) {
            System.out.println(SEPARATOR);
            System.out.println("Access denied. Returning to role
selection.");
            System.out.println(SEPARATOR);
            return;
        }

        runAdminMenu();
    }

    private boolean attemptLogin() {
        int attempts = 0;
        while (attempts < MAX_ATTEMPTS) {
            System.out.print("Enter admin password: ");
            String input = scanner.nextLine().trim();
            if (input.equals(ADMIN_PASSWORD)) {
                System.out.println("Access granted. Welcome, Admin.");
                return true;
            }
            attempts++;
            int remaining = MAX_ATTEMPTS - attempts;
            if (remaining > 0) {
                System.out.println("Incorrect password. " + remaining + "
attempt(s) remaining.");
            }
        }
        return false;
    }

    private void runAdminMenu() {
        boolean adminRunning = true;
        while (adminRunning) {
            showAdminMenu();
            String choice = scanner.nextLine().trim();
            adminRunning = handleAdminChoice(choice);
        }
    }

    private void showAdminMenu() {
        System.out.println("\n" + SEPARATOR);
        System.out.println("  ADMIN MENU");
        System.out.println(SEPARATOR);
        System.out.println("1. View active configuration");
        System.out.println("2. Update a base fare rule");
        System.out.println("3. Update a passenger discount");
        System.out.println("4. Update a daily cap");
        System.out.println("5. Return to role selection");
        System.out.print("Enter choice: ");
    }

    private boolean handleAdminChoice(String choice) {
        switch (choice) {
            case "1":
```

```
                configManager.displayConfig();
                return true;
            case "2":
                handleUpdateFare();
                return true;
            case "3":
                handleUpdateDiscount();
                return true;
            case "4":
                handleUpdateCap();
                return true;
            case "5":
                return false;
            default:
                System.out.println("Invalid option. Please enter 1-5.");
                return true;
        }
    }

    private void handleUpdateFare() {
        System.out.print("From zone (1-5): ");
        int fromZone = readInt();
        System.out.print("To zone (1-5): ");
        int toZone = readInt();
        System.out.print("Time band (PEAK / OFF_PEAK): ");
        String timeBand = scanner.nextLine().trim().toUpperCase();
        System.out.print("New fare (e.g. 3.50): ");
        BigDecimal newFare = readBigDecimal();

        boolean updated = configManager.updateBaseFare(fromZone, toZone,
timeBand, newFare);
        if (updated) {
            System.out.println("Fare rule updated successfully.");
        }
    }

    private void handleUpdateDiscount() {
        System.out.print("Passenger type
(ADULT/STUDENT/CHILD/SENIOR_CITIZEN): ");
        String type = scanner.nextLine().trim().toUpperCase();
        System.out.print("New discount rate (0.00-1.00, e.g. 0.25 for 25%):
");
        BigDecimal rate = readBigDecimal();

        boolean updated = configManager.updateDiscountRate(type, rate);
        if (updated) {
            System.out.println("Discount rate updated successfully.");
        }
    }

    private void handleUpdateCap() {
        System.out.print("Passenger type
(ADULT/STUDENT/CHILD/SENIOR_CITIZEN): ");
        String type = scanner.nextLine().trim().toUpperCase();
        System.out.print("New daily cap (e.g. 8.00): ");
        BigDecimal cap = readBigDecimal();
```

```
        boolean updated = configManager.updateDailyCap(type, cap);
        if (updated) {
            System.out.println("Daily cap updated successfully.");
        }
    }

    private int readInt() {
        int value = 0;
        boolean valid = false;
        while (!valid) {
            try {
                value = Integer.parseInt(scanner.nextLine().trim());
                valid = true;
            } catch (NumberFormatException e) {
                System.out.print("Please enter a whole number: ");
            }
        }
        return value;
    }

    private BigDecimal readBigDecimal() {
        BigDecimal value = BigDecimal.ZERO;
        boolean valid = false;
        while (!valid) {
            try {
                value = new BigDecimal(scanner.nextLine().trim());
                valid = true;
            } catch (NumberFormatException e) {
                System.out.print("Please enter a valid decimal number (e.g.
3.50): ");
            }
        }
        return value;
    }
}
```

CityRideDataset.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.HashMap;
import java.util.Map;

final class CityRideDataset {

    private CityRideDataset() {}

    public static final int MIN_ZONE = 1;
    public static final int MAX_ZONE = 5;

    public enum TimeBand {
        PEAK,
        OFF_PEAK
    }

    public enum PassengerType {
```

```
        ADULT,
        STUDENT,
        CHILD,
        SENIOR_CITIZEN
    }

    public static final Map<PassengerType, BigDecimal> DISCOUNT_RATE =
Map.of(
            PassengerType.ADULT, new BigDecimal("0.00"),
            PassengerType.STUDENT, new BigDecimal("0.25"),
            PassengerType.CHILD, new BigDecimal("0.50"),
            PassengerType.SENIOR_CITIZEN, new BigDecimal("0.30")
    );

    public static final Map<PassengerType, BigDecimal> DAILY_CAP = Map.of(
            PassengerType.ADULT, new BigDecimal("8.00"),
            PassengerType.STUDENT, new BigDecimal("6.00"),
            PassengerType.CHILD, new BigDecimal("4.00"),
            PassengerType.SENIOR_CITIZEN, new BigDecimal("7.00")
    );

    public static final Map<String, BigDecimal> BASE_FARE =
buildBaseFare();

    public static BigDecimal getBaseFare(int fromZone, int toZone, TimeBand
timeBand) {
        return BASE_FARE.get(key(fromZone, toZone, timeBand));
    }

    public static String key(int fromZone, int toZone, TimeBand timeBand) {
        return fromZone + "-" + toZone + "-" + timeBand.name();
    }

    private static BigDecimal money(String amount) {
        return new BigDecimal(amount).setScale(2, RoundingMode.HALF_UP);
    }

    private static Map<String, BigDecimal> buildBaseFare() {
        Map<String, BigDecimal> m = new HashMap<>();

        put(m,1,1,TimeBand.PEAK,"2.50"); put(m,1,2,TimeBand.PEAK,"3.20");
        put(m,1,3,TimeBand.PEAK,"3.80"); put(m,1,4,TimeBand.PEAK,"4.40");
        put(m,1,5,TimeBand.PEAK,"5.00");

        put(m,2,1,TimeBand.PEAK,"3.20"); put(m,2,2,TimeBand.PEAK,"2.30");
        put(m,2,3,TimeBand.PEAK,"3.10"); put(m,2,4,TimeBand.PEAK,"3.80");
        put(m,2,5,TimeBand.PEAK,"4.50");

        put(m,3,1,TimeBand.PEAK,"3.80"); put(m,3,2,TimeBand.PEAK,"3.10");
        put(m,3,3,TimeBand.PEAK,"2.10"); put(m,3,4,TimeBand.PEAK,"3.00");
        put(m,3,5,TimeBand.PEAK,"3.70");

        put(m,4,1,TimeBand.PEAK,"4.40"); put(m,4,2,TimeBand.PEAK,"3.80");
        put(m,4,3,TimeBand.PEAK,"3.00"); put(m,4,4,TimeBand.PEAK,"2.00");
        put(m,4,5,TimeBand.PEAK,"2.90");

        put(m,5,1,TimeBand.PEAK,"5.00"); put(m,5,2,TimeBand.PEAK,"4.50");
```

```
        put(m,5,3,TimeBand.PEAK,"3.70"); put(m,5,4,TimeBand.PEAK,"2.90");
        put(m,5,5,TimeBand.PEAK,"1.90");

        put(m,1,1,TimeBand.OFF_PEAK,"2.00");
put(m,1,2,TimeBand.OFF_PEAK,"2.70");
        put(m,1,3,TimeBand.OFF_PEAK,"3.20");
put(m,1,4,TimeBand.OFF_PEAK,"3.70");
        put(m,1,5,TimeBand.OFF_PEAK,"4.20");

        put(m,2,1,TimeBand.OFF_PEAK,"2.70");
put(m,2,2,TimeBand.OFF_PEAK,"1.90");
        put(m,2,3,TimeBand.OFF_PEAK,"2.60");
put(m,2,4,TimeBand.OFF_PEAK,"3.20");
        put(m,2,5,TimeBand.OFF_PEAK,"3.80");

        put(m,3,1,TimeBand.OFF_PEAK,"3.20");
put(m,3,2,TimeBand.OFF_PEAK,"2.60");
        put(m,3,3,TimeBand.OFF_PEAK,"1.70");
put(m,3,4,TimeBand.OFF_PEAK,"2.50");
        put(m,3,5,TimeBand.OFF_PEAK,"3.10");

        put(m,4,1,TimeBand.OFF_PEAK,"3.70");
put(m,4,2,TimeBand.OFF_PEAK,"3.20");
        put(m,4,3,TimeBand.OFF_PEAK,"2.50");
put(m,4,4,TimeBand.OFF_PEAK,"1.60");
        put(m,4,5,TimeBand.OFF_PEAK,"2.40");

        put(m,5,1,TimeBand.OFF_PEAK,"4.20");
put(m,5,2,TimeBand.OFF_PEAK,"3.80");
        put(m,5,3,TimeBand.OFF_PEAK,"3.10");
put(m,5,4,TimeBand.OFF_PEAK,"2.40");
        put(m,5,5,TimeBand.OFF_PEAK,"1.50");

        return Map.copyOf(m);
    }

    private static void put(Map<String, BigDecimal> m, int from, int to,
TimeBand band, String amount) {
        m.put(key(from, to, band), money(amount));
    }
}
```

CityRideLite.java

```
import java.util.Scanner;

// CityRideLite Class
public class CityRideLite {

    private static final String SEPARATOR = "    ====================";

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        ConfigManager configManager = new ConfigManager();
        configManager.loadFromFile("config.json");
```

```
        showBanner();

        boolean running = true;
        while (running) {
            showRoleMenu();
            String input = scanner.nextLine().trim().toUpperCase();

            if (input.equals("RIDER")) {
                RiderSession riderSession = new RiderSession(scanner,
configManager);
                riderSession.start();

            } else if (input.equals("ADMIN")) {
                AdminSession adminSession = new AdminSession(scanner,
configManager);
                adminSession.start();

            } else if (input.equals("EXIT")) {
                running = false;

            } else {
                System.out.println("Invalid choice. Please enter RIDER,
ADMIN, or EXIT.");
            }
        }

        System.out.println(SEPARATOR);
        System.out.println("Thank you for using CityRide Lite. Goodbye!");
        System.out.println(SEPARATOR);
        scanner.close();
    }

    private static void showBanner() {
        System.out.println(" ");
        System.out.println(SEPARATOR);
        System.out.println("   WELCOME TO CITYRIDE LITE");
        System.out.println("   Transport Fare Companion");
        System.out.println(SEPARATOR);
    }

    private static void showRoleMenu() {
        System.out.println("\nSelect role:");
        System.out.println("  -----------------------------");
        System.out.println("  RIDER  - Manage your journeys");
        System.out.println("  ADMIN  - System configuration");
        System.out.println("  EXIT   - Quit the program");
        System.out.println("  -----------------------------");
        System.out.print("Enter role: ");
    }
}
```

ConfigManager.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;
```

```
import java.util.HashMap;
import java.util.Map;

// ConfigManager Class
public class ConfigManager {

    private Map<String, BigDecimal> baseFares;

    private Map<String, BigDecimal> discountRates;

    private Map<String, BigDecimal> dailyCaps;

    public ConfigManager() {
        loadDefaults();
    }

    public void loadFromFile(String filePath) {
        java.io.File configFile = new java.io.File(filePath);
        if (!configFile.exists()) {
            System.out.println("[Config] config.json not found. Using
default values.");
            return;
        }
        System.out.println("[Config] Configuration loaded from " +
filePath);
    }

    private void loadDefaults() {
        loadDefaultFaresPeak();
        loadDefaultFaresOffPeak();
        loadDefaultDiscounts();
        loadDefaultCaps();
    }

    private void loadDefaultFaresPeak() {
        baseFares = new HashMap<>();
        putFare(1, 1, "PEAK", "2.50"); putFare(1, 2, "PEAK", "3.20");
        putFare(1, 3, "PEAK", "3.80"); putFare(1, 4, "PEAK", "4.40");
        putFare(1, 5, "PEAK", "5.00");
        putFare(2, 1, "PEAK", "3.20"); putFare(2, 2, "PEAK", "2.30");
        putFare(2, 3, "PEAK", "3.10"); putFare(2, 4, "PEAK", "3.80");
        putFare(2, 5, "PEAK", "4.50");
        putFare(3, 1, "PEAK", "3.80"); putFare(3, 2, "PEAK", "3.10");
        putFare(3, 3, "PEAK", "2.10"); putFare(3, 4, "PEAK", "3.00");
        putFare(3, 5, "PEAK", "3.70");
        putFare(4, 1, "PEAK", "4.40"); putFare(4, 2, "PEAK", "3.80");
        putFare(4, 3, "PEAK", "3.00"); putFare(4, 4, "PEAK", "2.00");
        putFare(4, 5, "PEAK", "2.90");
        putFare(5, 1, "PEAK", "5.00"); putFare(5, 2, "PEAK", "4.50");
        putFare(5, 3, "PEAK", "3.70"); putFare(5, 4, "PEAK", "2.90");
        putFare(5, 5, "PEAK", "1.90");
    }

    private void loadDefaultFaresOffPeak() {
        putFare(1, 1, "OFF_PEAK", "2.00"); putFare(1, 2, "OFF_PEAK",
"2.70");
        putFare(1, 3, "OFF_PEAK", "3.20"); putFare(1, 4, "OFF_PEAK",
```

```
"3.70");
        putFare(1, 5, "OFF_PEAK", "4.20");
        putFare(2, 1, "OFF_PEAK", "2.70"); putFare(2, 2, "OFF_PEAK",
"1.90");
        putFare(2, 3, "OFF_PEAK", "2.60"); putFare(2, 4, "OFF_PEAK",
"3.20");
        putFare(2, 5, "OFF_PEAK", "3.80");
        putFare(3, 1, "OFF_PEAK", "3.20"); putFare(3, 2, "OFF_PEAK",
"2.60");
        putFare(3, 3, "OFF_PEAK", "1.70"); putFare(3, 4, "OFF_PEAK",
"2.50");
        putFare(3, 5, "OFF_PEAK", "3.10");
        putFare(4, 1, "OFF_PEAK", "3.70"); putFare(4, 2, "OFF_PEAK",
"3.20");
        putFare(4, 3, "OFF_PEAK", "2.50"); putFare(4, 4, "OFF_PEAK",
"1.60");
        putFare(4, 5, "OFF_PEAK", "2.40");
        putFare(5, 1, "OFF_PEAK", "4.20"); putFare(5, 2, "OFF_PEAK",
"3.80");
        putFare(5, 3, "OFF_PEAK", "3.10"); putFare(5, 4, "OFF_PEAK",
"2.40");
        putFare(5, 5, "OFF_PEAK", "1.50");
    }

    private void loadDefaultDiscounts() {
        discountRates = new HashMap<>();
        discountRates.put("ADULT", new BigDecimal("0.00"));
        discountRates.put("STUDENT", new BigDecimal("0.25"));
        discountRates.put("CHILD", new BigDecimal("0.50"));
        discountRates.put("SENIOR_CITIZEN", new BigDecimal("0.30"));
    }

    private void loadDefaultCaps() {
        dailyCaps = new HashMap<>();
        dailyCaps.put("ADULT", new BigDecimal("8.00"));
        dailyCaps.put("STUDENT", new BigDecimal("6.00"));
        dailyCaps.put("CHILD", new BigDecimal("4.00"));
        dailyCaps.put("SENIOR_CITIZEN", new BigDecimal("7.00"));
    }

    public BigDecimal getBaseFare(int fromZone, int toZone, String
timeBand) {
        String key = buildFareKey(fromZone, toZone, timeBand);
        return baseFares.getOrDefault(key, BigDecimal.ZERO);
    }

    public BigDecimal getDiscountRate(String passengerType) {
        return discountRates.getOrDefault(passengerType.toUpperCase(),
BigDecimal.ZERO);
    }

    public BigDecimal getDailyCap(String passengerType) {
        return dailyCaps.getOrDefault(passengerType.toUpperCase(), new
BigDecimal("99.99"));
    }

    public boolean updateBaseFare(int fromZone, int toZone, String
```

```
timeBand, BigDecimal newFare) {
        if (newFare.compareTo(BigDecimal.ZERO) <= 0) {
            System.out.println("Error: Fare must be greater than zero.");
            return false;
        }
        String key = buildFareKey(fromZone, toZone, timeBand);
        baseFares.put(key, newFare.setScale(2, RoundingMode.HALF_UP));
        return true;
    }

    public boolean updateDiscountRate(String passengerType, BigDecimal
newRate) {
        if (newRate.compareTo(BigDecimal.ZERO) < 0 ||
newRate.compareTo(BigDecimal.ONE) > 0) {
            System.out.println("Error: Discount rate must be between 0.00
and 1.00.");
            return false;
        }
        if (!discountRates.containsKey(passengerType.toUpperCase())) {
            System.out.println("Error: Unknown passenger type: " +
passengerType);
            return false;
        }
        discountRates.put(passengerType.toUpperCase(), newRate);
        return true;
    }

    public boolean updateDailyCap(String passengerType, BigDecimal newCap)
{
        if (newCap.compareTo(BigDecimal.ZERO) <= 0) {
            System.out.println("Error: Daily cap must be greater than
zero.");
            return false;
        }
        if (!dailyCaps.containsKey(passengerType.toUpperCase())) {
            System.out.println("Error: Unknown passenger type: " +
passengerType);
            return false;
        }
        dailyCaps.put(passengerType.toUpperCase(), newCap.setScale(2,
RoundingMode.HALF_UP));
        return true;
    }

    public void displayConfig() {
        System.out.println("--------------------");
        System.out.println("DISCOUNT RATES:");
        for (Map.Entry<String, BigDecimal> entry :
discountRates.entrySet()) {
            System.out.println("  " + entry.getKey() + ": "
                    + entry.getValue().multiply(new BigDecimal("100")) +
"%");
        }
        System.out.println("DAILY CAPS:");
        for (Map.Entry<String, BigDecimal> entry : dailyCaps.entrySet()) {
            System.out.println("  " + entry.getKey() + ": £" +
entry.getValue());
```

```
        }
        System.out.println("--------------------");
    }

    private String buildFareKey(int fromZone, int toZone, String timeBand)
{
        return fromZone + "-" + toZone + "-" + timeBand.toUpperCase();
    }

    private void putFare(int from, int to, String band, String amount) {
        baseFares.put(buildFareKey(from, to, band),
                new BigDecimal(amount).setScale(2, RoundingMode.HALF_UP));
    }
}
```

FareCalculator.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;

// FareCalculator Class
public class FareCalculator {

    private final ConfigManager configManager;

    public FareCalculator(ConfigManager configManager) {
        this.configManager = configManager;
    }

    public BigDecimal[] calculate(int fromZone, int toZone, String
timeBand, String passengerType,
                                  BigDecimal totalSoFar) {

        BigDecimal baseFare = configManager.getBaseFare(fromZone, toZone,
timeBand);
        BigDecimal discountRate =
configManager.getDiscountRate(passengerType);
        BigDecimal dailyCap = configManager.getDailyCap(passengerType);

        BigDecimal discountApplied = baseFare.multiply(discountRate)
.setScale(2,
                RoundingMode.HALF_UP);

        BigDecimal fareAfterDiscount = baseFare.subtract(discountApplied);

        BigDecimal chargedFare = applyDailyCap(fareAfterDiscount,
totalSoFar, dailyCap);

        return new BigDecimal[]{
                baseFare, discountApplied, chargedFare
        };
    }

    public BigDecimal getDailyCap(String passengerType) {
        return configManager.getDailyCap(passengerType);
```

```
    }

    private BigDecimal applyDailyCap(BigDecimal fare, BigDecimal
totalSoFar, BigDecimal cap) {

        BigDecimal projectedTotal = totalSoFar.add(fare);
        if (projectedTotal.compareTo(cap) > 0) {
            BigDecimal remaining = cap.subtract(totalSoFar);
            if (remaining.compareTo(BigDecimal.ZERO) < 0) {
                return BigDecimal.ZERO;
            }
            return remaining;
        }
        return fare;
    }
}
```

FileManager.java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;

// FileManager Class
public class FileManager {

    private static final String CSV_HEADER =
            "ID,Date,PassengerName,FromZone,ToZone,TimeBand,PassengerType,"
                    + "BaseFare,DiscountApplied,ChargedFare";

    private static final int CSV_COLUMN_COUNT = 10;

    public void saveProfile(RiderProfile profile, String filePath) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(filePath))) {
            writer.write("{");
            writer.newLine();
            writer.write("  \"name\": \"" + profile.getName() + "\",");
            writer.newLine();
            writer.write("  \"passengerType\": \"" +
profile.getPassengerType() + "\",");
            writer.newLine();
            writer.write("  \"defaultPayment\": \"" +
profile.getDefaultPayment() + "\"");
            writer.newLine();
            writer.write("}");
            System.out.println("Profile saved to " + filePath);
        } catch (IOException e) {
            System.out.println("Error saving profile: " + e.getMessage());
```

```
        }
    }

    public RiderProfile loadProfile(String filePath) {
        File profileFile = new File(filePath);
        if (!profileFile.exists()) {
            System.out.println("Profile file not found: " + filePath);
            return null;
        }
        return parseProfileFile(filePath);
    }

    private RiderProfile parseProfileFile(String filePath) {
        String name           = "";
        String passengerType  = "";
        String defaultPayment = "";

        try (BufferedReader reader = new BufferedReader(new
FileReader(filePath))) {
            String line;
            while ((line = reader.readLine()) != null) {
                line = line.trim();
                if (line.contains("\"name\"")) {
                    name = extractJsonValue(line);
                } else if (line.contains("\"passengerType\"")) {
                    passengerType = extractJsonValue(line);
                } else if (line.contains("\"defaultPayment\"")) {
                    defaultPayment = extractJsonValue(line);
                }
            }
            System.out.println("Profile loaded for: " + name);
            return new RiderProfile(name, passengerType, defaultPayment);
        } catch (IOException e) {
            System.out.println("Error loading profile: " + e.getMessage());
            return null;
        }
    }

    public void exportJourneysCsv(List<Journey> journeys, String riderName)
{
        String fileName = buildFileName(riderName, "journeys", ".csv");
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName))) {
            writer.write(CSV_HEADER);
            writer.newLine();
            for (Journey journey : journeys) {
                writer.write(journey.toCsvRow());
                writer.newLine();
            }
            System.out.println("Journeys exported to " + fileName);
        } catch (IOException e) {
            System.out.println("Error exporting journeys: " +
e.getMessage());
        }
    }

    public List<String[]> importJourneysCsv(String filePath) {
```

```
        List<String[]> rows = new ArrayList<>();
        File importFile = new File(filePath);
        if (!importFile.exists()) {
            System.out.println("Import file not found: " + filePath);
            return rows;
        }
        return readCsvRows(filePath, rows);
    }

    private List<String[]> readCsvRows(String filePath, List<String[]>
rows) {
        int imported = 0;
        int rejected = 0;
        try (BufferedReader reader = new BufferedReader(new
FileReader(filePath))) {
            reader.readLine(); // skip header row
            String line;
            while ((line = reader.readLine()) != null) {
                String[] parts = line.split(",");
                if (parts.length != CSV_COLUMN_COUNT) {
                    rejected++;
                } else {
                    rows.add(parts);
                    imported++;
                }
            }
        } catch (IOException e) {
            System.out.println("Error reading import file: " +
e.getMessage());
        }
        System.out.println("Import complete: " + imported
                + " imported, " + rejected + " rejected.");
        return rows;
    }

    public void exportSummaryText(String summaryText, String riderName) {
        String fileName = buildFileName(riderName, "summary", ".txt");
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName))) {
            writer.write(summaryText);
            System.out.println("Summary saved to " + fileName);
        } catch (IOException e) {
            System.out.println("Error saving summary: " + e.getMessage());
        }
    }

    public void exportSummaryCsv(List<Journey> journeys, String riderName)
{
        String fileName = buildFileName(riderName, "summary", ".csv");
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName))) {
            writer.write(CSV_HEADER);
            writer.newLine();
            for (Journey journey : journeys) {
                writer.write(journey.toCsvRow());
                writer.newLine();
            }
```

```
            System.out.println("Summary CSV saved to " + fileName);
        } catch (IOException e) {
            System.out.println("Error saving summary CSV: " +
e.getMessage());
        }
    }

    private String buildFileName(String riderName, String type, String
extension) {
        String date     =
LocalDate.now().format(DateTimeFormatter.ofPattern("dd-MM-yyyy"));
        String safeName = riderName.replaceAll("\\s+", "_");
        return safeName + "_" + type + "_" + date + extension;
    }

    private String extractJsonValue(String line) {
        int colonIndex  = line.indexOf(":");
        int valueStart  = line.indexOf("\"", colonIndex) + 1;
        int valueEnd    = line.lastIndexOf("\"");
        if (valueStart > 0 && valueEnd > valueStart) {
            return line.substring(valueStart, valueEnd);
        }
        return "";
    }
}
```

Journey.java

```
import java.math.BigDecimal;

// Journey Class
public class Journey {

    private int id;
    private String date;
    private String passengerName;
    private int fromZone;
    private int toZone;
    private String timeBand;
    private String passengerType;
    private BigDecimal baseFare;
    private BigDecimal discountApplied;
    private BigDecimal chargedFare;

    public Journey(int id, String date, String passengerName,
                   int fromZone, int toZone,
                   String timeBand, String passengerType,
                   BigDecimal baseFare, BigDecimal discountApplied,
                   BigDecimal chargedFare) {
        this.id = id;
        this.date = date;
        this.passengerName = passengerName;
        this.fromZone = fromZone;
        this.toZone = toZone;
        this.timeBand = timeBand;
        this.passengerType = passengerType;
```

```
        this.baseFare = baseFare;
        this.discountApplied = discountApplied;
        this.chargedFare = chargedFare;
    }

    public int getId() { return id; }

    public String getDate() { return date; }

    public String getPassengerName() { return passengerName; }

    public int getFromZone() { return fromZone; }

    public int getToZone() { return toZone; }

    public String getTimeBand() { return timeBand; }

    public String getPassengerType() { return passengerType; }

    public int getZonesCrossed() { return Math.abs(toZone - fromZone); }

    public BigDecimal getBaseFare() { return baseFare; }

    public BigDecimal getDiscountApplied() { return discountApplied; }

    public BigDecimal getChargedFare() { return chargedFare; }

    public void setChargedFare(BigDecimal chargedFare) {
        this.chargedFare = chargedFare;
    }

    public void setDiscountApplied(BigDecimal discountApplied) {
        this.discountApplied = discountApplied;
    }

    public void displayJourney() {
        System.out.println("--------------------");
        System.out.println("Journey ID     : " + id);
        System.out.println("Date           : " + date);
        System.out.println("Passenger      : " + passengerName);
        System.out.println("From Zone      : " + fromZone);
        System.out.println("To Zone        : " + toZone);
        System.out.println("Zones Crossed  : " + getZonesCrossed());
        System.out.println("Time Band      : " + timeBand);
        System.out.println("Passenger Type : " + passengerType);
        System.out.println("Base Fare      : £" + baseFare);
        System.out.println("Discount       : £" + discountApplied);
        System.out.println("Charged Fare   : £" + chargedFare);
        System.out.println("--------------------");
    }

    public String toCsvRow() {
        return id + "," + date + "," + passengerName + ","
                + fromZone + "," + toZone + ","
                + timeBand + "," + passengerType + ","
                + baseFare + "," + discountApplied + "," + chargedFare;
```

```
    }
}
```

JourneyManager.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

// JourneyManager Class
public class JourneyManager {

    private final List<Journey> journeys;
    private final FareCalculator fareCalculator;
    private int nextId;

    private static final String SEPARATOR = "--------------------";
    private static final String WIDE_SEP  =
"========================================";
    private static final DateTimeFormatter DATE_FORMAT =
            DateTimeFormatter.ofPattern("dd/MM/yyyy");

    public JourneyManager(FareCalculator fareCalculator) {
        this.journeys = new ArrayList<>();
        this.fareCalculator = fareCalculator;
        this.nextId = 1;
    }

    public void addJourney(String passengerName, int fromZone, int toZone,
                           String timeBand, String passengerType) {

        BigDecimal totalSoFar = getTotalChargedForType(passengerType);
        BigDecimal[] fareResult = fareCalculator.calculate(
                fromZone, toZone, timeBand, passengerType, totalSoFar);

        String today = LocalDate.now().format(DATE_FORMAT);
        Journey journey = new Journey(nextId, today, passengerName,
                fromZone, toZone, timeBand, passengerType,
                fareResult[0], fareResult[1], fareResult[2]);

        journeys.add(journey);
        nextId++;
        printAddConfirmation(fareResult);
    }

    public void removeJourney(int id) {
        int index = findIndexById(id);
        if (index == -1) {
```

```
            System.out.println(SEPARATOR);
            System.out.println("Error: Journey ID " + id + " not found.");
            System.out.println(SEPARATOR);
            return;
        }
        journeys.remove(index);
        recalculateTotals();
        System.out.println(SEPARATOR);
        System.out.println("Journey " + id + " removed. Totals
recalculated.");
        System.out.println(SEPARATOR);
    }

    public void recalculateTotals() {
        Map<String, BigDecimal> runningTotals = new HashMap<>();
        for (Journey journey : journeys) {
            String type = journey.getPassengerType();
            BigDecimal totalSoFar = runningTotals.getOrDefault(type,
BigDecimal.ZERO);
            BigDecimal[] fareResult = fareCalculator.calculate(
                    journey.getFromZone(), journey.getToZone(),
                    journey.getTimeBand(), type, totalSoFar);
            journey.setDiscountApplied(fareResult[1]);
            journey.setChargedFare(fareResult[2]);
            runningTotals.put(type, totalSoFar.add(fareResult[2]));
        }
    }

    public void listJourneys() {
        if (journeys.isEmpty()) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys recorded.");
            System.out.println(SEPARATOR);
            return;
        }
        for (Journey journey : journeys) {
            journey.displayJourney();
        }
    }

    public void filterByPassengerType(String passengerType) {
        boolean found = false;
        for (Journey journey : journeys) {
            if (journey.getPassengerType().equalsIgnoreCase(passengerType))
{
                journey.displayJourney();
                found = true;
            }
        }
        if (!found) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys found for type: " +
passengerType);
            System.out.println(SEPARATOR);
        }
    }
```

```
    public void showRunningTotals() {
        if (journeys.isEmpty()) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys recorded yet.");
            System.out.println(SEPARATOR);
            return;
        }
        Set<String> types = collectPassengerTypes();
        System.out.println(SEPARATOR);
        System.out.println("RUNNING TOTALS:");
        for (String type : types) {
            BigDecimal total = getTotalChargedForType(type);
            BigDecimal cap   = fareCalculator.getDailyCap(type);
            System.out.println("  " + type + ": £" + total + " / £" + cap +
" cap");
        }
        System.out.println(SEPARATOR);
    }

    public void showSummary() {
        if (journeys.isEmpty()) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys to summarise.");
            System.out.println(SEPARATOR);
            return;
        }
        int[]        counts  = countTimeBands();
        BigDecimal[] totals  = calculateTotals();
        int          maxId   = findMostExpensiveId();

        BigDecimal totalCharged = totals[0];
        BigDecimal totalUncapped = totals[1];
        BigDecimal average = totalCharged.divide(
                BigDecimal.valueOf(journeys.size()), 2,
RoundingMode.HALF_UP);
        BigDecimal savings = totalUncapped.subtract(totalCharged);

        printSummary(totalCharged, average, maxId, savings, counts[0],
counts[1]);
    }

    public boolean hasJourneys() {
        return !journeys.isEmpty();
    }

    public List<Journey> getJourneys() {
        return Collections.unmodifiableList(journeys);
    }

    private BigDecimal getTotalChargedForType(String passengerType) {
        BigDecimal total = BigDecimal.ZERO;
        for (Journey journey : journeys) {
            if (journey.getPassengerType().equalsIgnoreCase(passengerType))
{
                total = total.add(journey.getChargedFare());
            }
        }
```

```
        return total;
    }

    private int findIndexById(int id) {
        for (int i = 0; i < journeys.size(); i++) {
            if (journeys.get(i).getId() == id) {
                return i;
            }
        }
        return -1;
    }

    private Set<String> collectPassengerTypes() {
        Set<String> types = new LinkedHashSet<>();
        for (Journey j : journeys) {
            types.add(j.getPassengerType());
        }
        return types;
    }

    private int[] countTimeBands() {
        int peak = 0;
        int offPeak = 0;
        for (Journey j : journeys) {
            if (j.getTimeBand().equalsIgnoreCase("PEAK")) {
                peak++;
            } else {
                offPeak++;
            }
        }
        return new int[]{peak, offPeak};
    }

    private BigDecimal[] calculateTotals() {
        BigDecimal totalCharged  = BigDecimal.ZERO;
        BigDecimal totalUncapped = BigDecimal.ZERO;
        for (Journey j : journeys) {
            totalCharged  = totalCharged.add(j.getChargedFare());
            totalUncapped = totalUncapped.add(
                    j.getBaseFare().subtract(j.getDiscountApplied()));
        }
        return new BigDecimal[]{totalCharged, totalUncapped};
    }

    private int findMostExpensiveId() {
        int maxId = -1;
        BigDecimal maxFare = BigDecimal.ZERO;
        for (Journey j : journeys) {
            if (j.getChargedFare().compareTo(maxFare) > 0) {
                maxFare = j.getChargedFare();
                maxId   = j.getId();
            }
        }
        return maxId;
    }

    private void printAddConfirmation(BigDecimal[] fareResult) {
```

```
        System.out.println(SEPARATOR);
        System.out.println("Journey added successfully.");
        System.out.println("Base fare    : £" + fareResult[0]);
        System.out.println("Discount     : £" + fareResult[1]);
        System.out.println("Charged fare : £" + fareResult[2]);
        System.out.println(SEPARATOR);
    }

    private void printSummary(BigDecimal total, BigDecimal average,
                              int maxId, BigDecimal savings,
                              int peakCount, int offPeakCount) {
        System.out.println(WIDE_SEP);
        System.out.println("  END OF DAY SUMMARY");
        System.out.println(WIDE_SEP);
        System.out.println("Total journeys         : " + journeys.size());
        System.out.println("Total cost             : £" + total);
        System.out.println("Average cost           : £" + average);
        System.out.println("Most expensive journey : ID " + maxId);
        System.out.println("Savings from cap       : £" + savings);
        System.out.println("Peak journeys          : " + peakCount);
        System.out.println("Off-peak journeys      : " + offPeakCount);
        System.out.println(WIDE_SEP);
    }
}
```

RiderProfile.java

```
// RiderProfile Class
public class RiderProfile {

    private String name;
    private String passengerType;
    private String defaultPayment;

    public RiderProfile(String name, String passengerType, String
defaultPayment) {
        this.name = name;
        this.passengerType = passengerType;
        this.defaultPayment = defaultPayment;
    }

    public String getName() {
        return name;
    }

    public String getPassengerType() {
        return passengerType;
    }

    public String getDefaultPayment() {
        return defaultPayment;
    }

    public void setName(String name) {
        this.name = name;
    }
```

```
    public void setPassengerType(String passengerType) {
        this.passengerType = passengerType;
    }

    public void setDefaultPayment(String defaultPayment) {
        this.defaultPayment = defaultPayment;
    }

    public void displayProfile() {
        System.out.println("--------------------");
        System.out.println("Name           : " + name);
        System.out.println("Passenger Type : " + passengerType);
        System.out.println("Default Payment: " + defaultPayment);
        System.out.println("--------------------");
    }
}
```

RiderSession.java

```
import java.util.List;
import java.util.Scanner;

// RiderSession Class
public class RiderSession {

    private static final String SEPARATOR    = "--------------------";
    private static final String WIDE_SEP     = "====================";

    private final Scanner scanner;
    private final ConfigManager configManager;
    private final FareCalculator fareCalculator;
    private final JourneyManager journeyManager;
    private final FileManager fileManager;
    private RiderProfile profile;

    public RiderSession(Scanner scanner, ConfigManager configManager) {
        this.scanner       = scanner;
        this.configManager = configManager;
        this.fareCalculator = new FareCalculator(configManager);
        this.journeyManager = new JourneyManager(fareCalculator);
        this.fileManager   = new FileManager();
    }

    public void start() {
        setupProfile();
        runRiderMenu();
    }

    private void setupProfile() {
        System.out.println(SEPARATOR);
        System.out.println("1. Load existing profile");
        System.out.println("2. Create new profile");
        System.out.println(SEPARATOR);
        System.out.print("Choose: ");
        String choice = scanner.nextLine().trim();
```

```
        if (choice.equals("1")) {
            loadExistingProfile();
        } else {
            createNewProfile();
        }
    }

    private void loadExistingProfile() {
        System.out.print("Enter profile filename (e.g.
deborah_profile.json): ");
        String filePath = scanner.nextLine().trim();
        profile = fileManager.loadProfile(filePath);
        if (profile == null) {
            System.out.println("Could not load profile. Creating a new one
instead.");
            createNewProfile();
        }
    }

    private void createNewProfile() {
        System.out.println(SEPARATOR);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine().trim();

        String passengerType = promptPassengerType();

        System.out.print("Default payment option (e.g. CARD, CASH): ");
        String payment = scanner.nextLine().trim().toUpperCase();

        profile = new RiderProfile(name, passengerType, payment);
        System.out.println("Profile created. Welcome, " + name + "!");
    }

    private void runRiderMenu() {
        boolean running = true;
        while (running) {
            showRiderMenu();
            String choice = scanner.nextLine().trim();
            running = handleRiderChoice(choice);
        }
    }

    private void showRiderMenu() {
        System.out.println("\n" + WIDE_SEP);
        System.out.println("  CITYRIDE LITE - " + profile.getName());
        System.out.println(WIDE_SEP);
        System.out.println("1.  View profile");
        System.out.println("2.  Add journey");
        System.out.println("3.  View all journeys");
        System.out.println("4.  Filter journeys by type");
        System.out.println("5.  Remove journey");
        System.out.println("6.  View running totals");
        System.out.println("7.  End-of-day summary");
        System.out.println("8.  Export journeys to CSV");
        System.out.println("9.  Export summary (CSV + text)");
        System.out.println("10. Import journeys from CSV");
```

```
        System.out.println("11. Save profile");
        System.out.println("12. Save and exit");
        System.out.print("Enter choice: ");
    }

    private boolean handleRiderChoice(String choice) {
        switch (choice) {
            case "1":  profile.displayProfile();           return true;
            case "2":  handleAddJourney();                 return true;
            case "3":  journeyManager.listJourneys();      return true;
            case "4":  handleFilterJourneys();             return true;
            case "5":  handleRemoveJourney();              return true;
            case "6":  journeyManager.showRunningTotals(); return true;
            case "7":  journeyManager.showSummary();       return true;
            case "8":  handleExportJourneys();             return true;
            case "9":  handleExportSummary();              return true;
            case "10": handleImportJourneys();             return true;
            case "11": handleSaveProfile();                return true;
            case "12": handleSaveAndExit();                return false;
            default:
                System.out.println("Invalid option. Please enter 1-12.");
                return true;
        }
    }

    private void handleAddJourney() {
        showFareReference();

        System.out.print("Passenger name (Enter to use profile name): ");
        String name = scanner.nextLine().trim();
        if (name.isEmpty()) {
            name = profile.getName();
        }

        int fromZone    = promptZone("from");
        int toZone      = promptZone("to");
        String timeBand = promptTimeBand();

        System.out.print("Passenger type [" + profile.getPassengerType()
                + "] (Enter to use profile): ");
        String typeInput     = scanner.nextLine().trim().toUpperCase();
        String passengerType = typeInput.isEmpty() ?
profile.getPassengerType() : typeInput;

        journeyManager.addJourney(name, fromZone, toZone, timeBand,
passengerType);
    }

    private void showFareReference() {
        System.out.println(SEPARATOR);
        System.out.println("FARE REFERENCE TABLE");
        System.out.println(SEPARATOR);
        System.out.println("Zone Pair | PEAK   | OFF_PEAK");
        System.out.println("----------+--------+---------");
        int[] zones = {1, 2, 3, 4, 5};
        for (int from : zones) {
            for (int to : zones) {
```

```
                String peak    = "£" + configManager.getBaseFare(from, to,
"PEAK");
                String offPeak = "£" + configManager.getBaseFare(from, to,
"OFF_PEAK");
                System.out.printf("  %d  →  %d  | %-6s | %s%n", from, to,
peak, offPeak);
            }
        }
        System.out.println(SEPARATOR);
        System.out.println("DISCOUNTS:");
        String[] types = {"ADULT", "STUDENT", "CHILD", "SENIOR_CITIZEN"};
        for (String t : types) {
            java.math.BigDecimal rate = configManager.getDiscountRate(t);
            java.math.BigDecimal cap  = configManager.getDailyCap(t);
            System.out.printf("  %-15s discount: %-4s  daily cap: £%s%n",
                    t,
                    rate.multiply(new
java.math.BigDecimal("100")).stripTrailingZeros().toPlainString() + "%",
                    cap);
        }
        System.out.println(SEPARATOR);
    }

    private void handleFilterJourneys() {
        String type = promptPassengerType();
        journeyManager.filterByPassengerType(type);
    }

    private void handleRemoveJourney() {
        if (!journeyManager.hasJourneys()) {
            System.out.println(SEPARATOR);
            System.out.println("No journeys to remove.");
            System.out.println(SEPARATOR);
            return;
        }
        journeyManager.listJourneys();
        System.out.print("Enter journey ID to remove: ");
        int id = readInt();
        journeyManager.removeJourney(id);
    }

    private void handleExportJourneys() {
        if (!journeyManager.hasJourneys()) {
            System.out.println("No journeys to export.");
            return;
        }
        fileManager.exportJourneysCsv(journeyManager.getJourneys(),
profile.getName());
    }

    private void handleExportSummary() {
        if (!journeyManager.hasJourneys()) {
            System.out.println("No journeys to export.");
            return;
        }
        journeyManager.showSummary();
        fileManager.exportSummaryCsv(journeyManager.getJourneys(),
```

```
profile.getName());
        fileManager.exportSummaryText(buildSummaryText(),
profile.getName());
    }

    private void handleImportJourneys() {
        System.out.print("Enter CSV filename to import (e.g. journeys.csv):
");
        String filePath = scanner.nextLine().trim();
        List<String[]> rows = fileManager.importJourneysCsv(filePath);

        int added = 0;
        for (String[] row : rows) {
            try {

                int from     = Integer.parseInt(row[3].trim());
                int to       = Integer.parseInt(row[4].trim());
                String band  = row[5].trim();
                String type  = row[6].trim();
                String name  = row[2].trim();
                journeyManager.addJourney(name, from, to, band, type);
                added++;
            } catch (Exception e) {

            }
        }
        System.out.println(added + " journey(s) added from import.");
    }

    private void handleSaveProfile() {
        String filePath = profile.getName().replaceAll("\\s+",
"_").toLowerCase()
                + "_profile.json";
        fileManager.saveProfile(profile, filePath);
    }

    private void handleSaveAndExit() {
        System.out.print("Save profile and journeys before exiting? (Y/N):
");
        String confirm = scanner.nextLine().trim().toUpperCase();
        if (confirm.equals("Y")) {
            handleSaveProfile();
            handleExportJourneys();
        }
        System.out.println(SEPARATOR);
        System.out.println("Session ended. Goodbye, " + profile.getName() +
"!");
        System.out.println(SEPARATOR);
    }

    private int promptZone(String label) {
        int zone = 0;
        boolean valid = false;
        while (!valid) {
            System.out.print("Enter " + label + " zone (1-5): ");
            zone = readInt();
            if (zone >= 1 && zone <= 5) {
```

```
                valid = true;
            } else {
                System.out.println("Zone must be between 1 and 5.");
            }
        }
        return zone;
    }

    private String promptTimeBand() {
        String band = "";
        boolean valid = false;
        while (!valid) {
            System.out.print("Time band (PEAK / OFF_PEAK): ");
            band = scanner.nextLine().trim().toUpperCase();
            if (band.equals("PEAK") || band.equals("OFF_PEAK")) {
                valid = true;
            } else {
                System.out.println("Please enter PEAK or OFF_PEAK.");
            }
        }
        return band;
    }

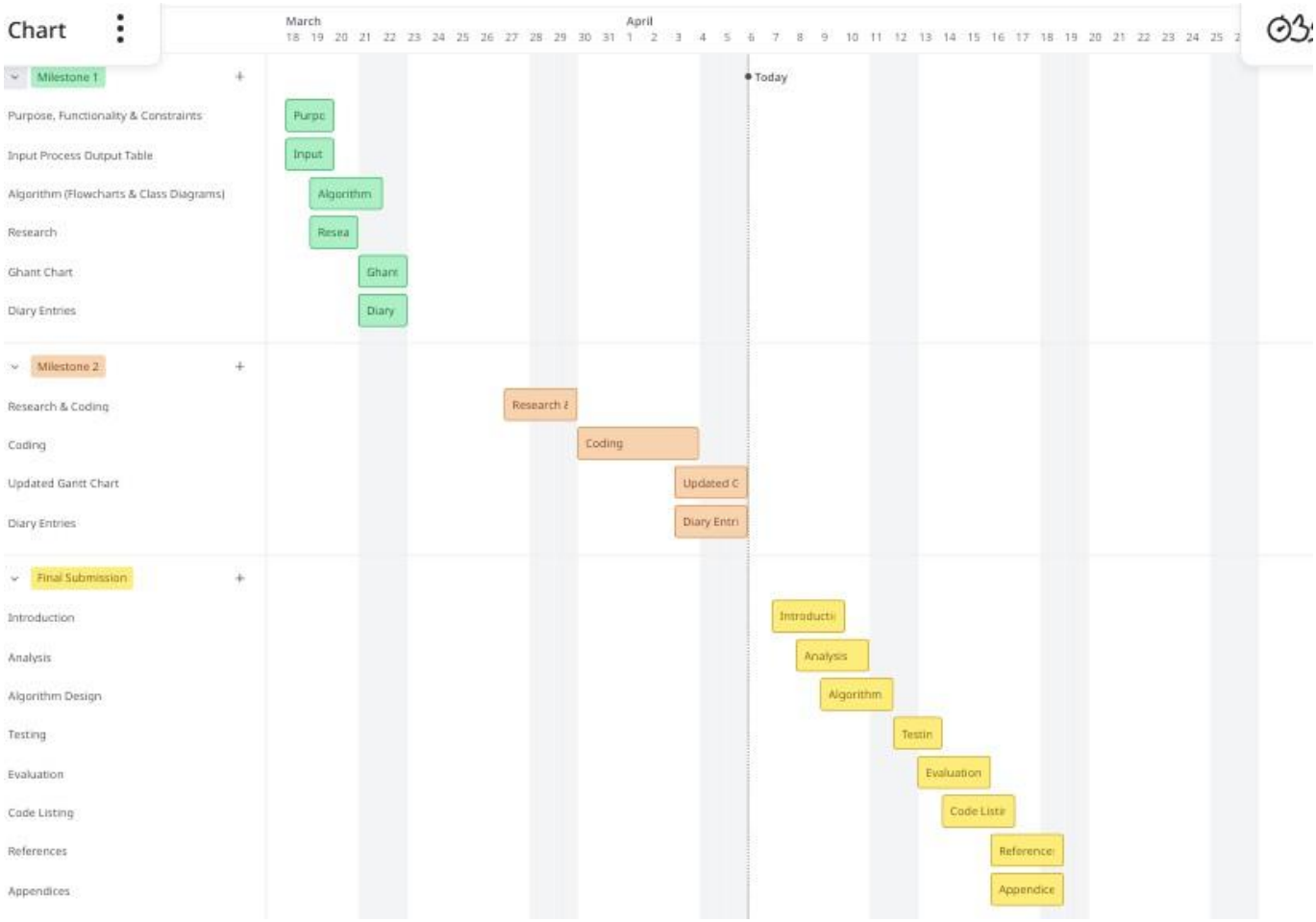
    private String promptPassengerType() {
        String type = "";
        boolean valid = false;
        while (!valid) {
            System.out.println("Passenger types: ADULT | STUDENT | CHILD |
SENIOR_CITIZEN");
            System.out.print("Enter type: ");
            type = scanner.nextLine().trim().toUpperCase();
            if (type.equals("ADULT") || type.equals("STUDENT")
                    || type.equals("CHILD") ||
type.equals("SENIOR_CITIZEN")) {
                valid = true;
            } else {
                System.out.println("Invalid type. Please try again.");
            }
        }
        return type;
    }

    private int readInt() {
        int value = 0;
        boolean valid = false;
        while (!valid) {
            try {
                value = Integer.parseInt(scanner.nextLine().trim());
                valid = true;
            } catch (NumberFormatException e) {
                System.out.print("Please enter a whole number: ");
            }
        }
        return value;
    }

    private String buildSummaryText() {
```

```
        return "CityRide Lite - End of Day Summary\n"
                + "Rider: " + profile.getName() + "\n"
                + "Date : " + java.time.LocalDate.now() + "\n"
                + "See CSV file for full journey breakdown.\n";
    }
}
```

Updated Gantt Chart



Diary Entries (at least 4)

Diary Entry 1 (29/03/2026 – Research and Initial Coding)

I began Part 2 by researching key topics: file handling from Oracle documentation, and ArrayList and HashMap from GeeksforGeeks. These were chosen to address major limitations from Part 1, especially fixed data storage. I started coding with the JourneyManager class because it connects most parts of the system. I replaced the fixed Journey[] array and counter with an ArrayList, removing the 5-journey limit. This also simplified methods like removeJourney() since ArrayList handles element shifting automatically. Overall, this change makes system more flexible and easier to maintain.

Diary Entry 2 (02/04/2026 – Continued Coding and Setup)

I developed the RiderProfile class to store user details such as name, passenger type, and payment option, using encapsulation with private fields and getters/setters. I then created the ConfigManager class, which stores fares, discounts, and caps using HashMaps instead of fixed values. It reads from a config file at startup, with fallback defaults from Part 1 if the file is missing. These foundation classes are important because other components depend on them. I encountered an error after switching to ArrayList because I forgot to update indexing in showSummary(). I fixed this by using a for-each loop, improving readability and consistency.

Diary Entry 3 (04/04/2026 – Completing part Coding and Integration)

I completed and integrated the remaining classes: FareCalculator, FileManager, AdminSession, RiderSession, and updated the main program. I separated fare calculation into FareCalculator to follow the Single Responsibility Principle, instead of combining it with journey management. The FileManager handles all file operations, including JSON profiles, CSV journeys, and summary reports, with error handling for each method. I implemented admin login with password protection and validation for updating fare rules. I tested the full system, including adding, deleting, and recalculating journeys, as well as admin updates. I also fixed issues like recalculating fares after deletions and handling invalid CSV rows safely.

Diary Entry 4 (06/04/2026 – Review and Professional Standards Check)

Before submission, I reviewed the code against professional programming standards from the NTIC Guide. I ensured correct naming conventions, short methods, and that each class has asingle responsibility. I checked that all fields are private, loops are properly controlled, and no method exceeds recommended length. I added Javadoc comments and inline explanations for complex logic like fare caps. I refactored showAdminMenu() by extracting logic into a helper method to improve structure. This review ensured the program is clean, maintainable, and meets assessment requirements.